

INFORME DE EJECUCIÓN DE PRUEBAS TDD

Sistema de Gestión Veterinaria

1. Información General

- **Proyecto:** Plataforma Web de Gestión Veterinaria
 - **Metodología:** Desarrollo Guiado por Pruebas (Test-Driven Development - TDD)
 - **Entorno Técnico:** Java 21+, Spring Boot 4, JUnit 5, Mockito, Spring Data JPA, PostgreSQL.
-

2. Resumen de Ejecución

El proyecto combina dos tipos de prueba. Las historias del bloque de cliente y de servicios usan pruebas unitarias con JUnit 5 y Mockito sobre la capa de servicio. Las historias del bloque de administración usan pruebas de integración con @SpringBootTest y MockMvc, que recorren la petición completa (seguridad, controlador, validación, servicio y base de datos H2 en memoria con el perfil test). En estas últimas, cada criterio de aceptación de Jira se tradujo en uno o más métodos de prueba cuyo @DisplayName referencia el criterio.

Tabla 1. Clases de prueba por historia de usuario

HU	Clase de prueba	Tipo	Pruebas	Responsable
HU-02	HU02AutenticacionTest	Integración	11	S. Velásquez
HU-21	HU21PanelAdministracionTest	Integración	6	S. Velásquez
HU-01	HU01RegistrarClienteTest	Integración	11	S. Velásquez
HU-06	HU06RegistrarVeterinarioTest	Integración	13	S. Velásquez
HU-11	HU11RegistrarMascotaTest	Integración	15	S. Velásquez
HU-12	HU12ConsultarMascotasTest	Integración	10	S. Velásquez
HU-16	HU16CrearServicioTest	Integración	15	S. Velásquez

HU	Clase de prueba	Tipo	Pruebas	Responsable
HU-22 (pantallas)	HU22MaestroProductosTest	Integración	13	S. Velásquez
HU-25	HU25AgendarCitaTest	Integración	14	S. Velásquez
HU-26	HU26ReporteCitasTest	Integración	13	S. Velásquez
HU-24	HU24HistoriaClinicaTest	Integración	10	S. Velásquez
HU-22	ProductoServiceTest	Unitaria	4	J. D. Cano
HU-17	HU17ModificarPrecioServicioTest	Unitaria	2	J. D. Cano
HU-18	HU18DesactivarServicioTest	Unitaria	3	J. D. Cano
HU-19	HU19ConsultarServiciosTest	Unitaria	3	J. D. Cano
HU-20	HU20ValidarDuracionServicioTest	Unitaria	4	J. D. Cano
HU-20 CA3	CitaServiceTest	Unitaria	2	J. D. Cano
HU-03	HU03PerfilClienteTest	Unitaria	2 (1 parametrizada)	J. D. Cano
HU-05	HU05CambiarPasswordTest	Unitaria	3 (1 parametrizada)	J. D. Cano
General	VeterinariaWebApplicationTests	Contexto	1	Equipo

La columna Pruebas cuenta los métodos anotados. Las dos pruebas parametrizadas generan varios casos cada una, por lo que la ejecución completa de la suite reporta 160 pruebas, todas en verde, al cierre de la entrega (26 de septiembre de 2026, 11:30).

Tabla 2. Resultados del ciclo rojo-verde en las pruebas de integración

Historia	Fase roja (ejecutadas / resultado)	Fase verde	Suite completa tras integrar
HU-02 y HU-21	18 / 3 fallos y 11 errores	18 / 0 fallos (tras una ejecución intermedia con 2 fallos)	18 pruebas en verde
HU-01	11 / 9 fallos	11 / 0 fallos	29 pruebas en verde
HU-06	No se conservó evidencia	13 / 0 fallos	42 pruebas en verde
HU-16	Error de compilación (clases inexistentes)	15 / 0 fallos	57 pruebas en verde
Integración con HU-17 a HU-20	No aplica	No aplica	70 pruebas, 5 fallos; luego 70 en verde (ver sección 4)
HU-25	14 / 11 fallos y 1 error	14 / 0 fallos	89 pruebas en verde
HU-11 y HU-12	25 / 21 fallos	25 / 0 fallos	114 pruebas en verde
HU-22 (pantallas)	13 / 10 fallos y 1 error	13 / 0 fallos	127 pruebas en verde
HU-26	13 / 12 fallos	13 / 0 fallos (tras ajuste de ruta)	150 pruebas en verde
HU-24	10 / 7 fallos	10 / 0 fallos	160 pruebas en verde

Las cifras incluyen solo las clases de cada historia, salvo en HU-02, HU-21 y HU-01, donde la ejecución incluyó además la prueba de arranque del contexto (VeterinariaWebApplicationTests). Desde HU-01, las pruebas que pasaron en fase roja corresponden a control de acceso: la configuración de Spring Security sobre /admin/** ya existía antes de cada historia. En HU-16 la fase roja se manifestó como error de compilación, porque la prueba referenciaba clases que aún no existían. Los casos particulares de HU-21, HU-26 y HU-24 se explican en sus secciones.

3. Registro Detallado de Pruebas TDD

Épica: Gestión de Productos

Historia de Usuario HU-22: Gestión de Catálogo de Medicamentos e Insumos

- **Como:** Administrador
 - **Quiero:** Registrar, clasificar y actualizar el catálogo de medicamentos e insumos especificando su precio, stock y tipo de aplicación.
 - **Para:** Permitir que los médicos veterinarios receten tratamientos durante las citas.
-

Criterio de Aceptación 1:

Dado que el administrador ingresa al formulario de nuevo medicamento con nombre comercial, presentación, precio unitario mayor a cero y stock disponible, **cuando** presiona el botón "Guardar Producto", **entonces** el sistema almacena el registro en la base de datos con estado ACTIVO, asigna un ID único y lo retorna para ser listado.

A. Código Fuente Involucrado

Código de la Prueba Unitaria (ProductoServiceTest.java):

```
@Test
@DisplayName("Registrar producto con estado ACTIVO, el sistema le asigna un ID único y lo devuelve")
void registrarProducto() {
    Producto productoCreado = new Producto();
    productoCreado.setIdProducto(1L);
    productoCreado.setNombre("NexGard Spectra");
    productoCreado.setPresentacion("Tableta masticable");
    productoCreado.setPrecioUnitario(72000.00);
    productoCreado.setStock(4);
    productoCreado.setEsRecetableVeterinario(true);
    productoCreado.setEstado(EstadoProducto.ACTIVO);

    when(productoRepository.save(any(Producto.class))).thenReturn(productoCreado);

    Producto resultado = productoService.registrarProducto(productoEntrada);

    assertNotNull(resultado, "El producto resultante no debe ser nulo");
    assertNotNull(resultado.getIdProducto(), "El producto resultante debe tener un ID asignado");
```

```

    assertEquals(EstadoProducto.ACTIVO, resultado.getEstado(), "El estado inicial debe ser ACTIVO");
    assertEquals(productoEntrada.getNombre(), resultado.getNombre(), "El nombre del producto debe coincidir");
}

```

Código de Producción Implementado (ProductoServiceImpl.java):

```

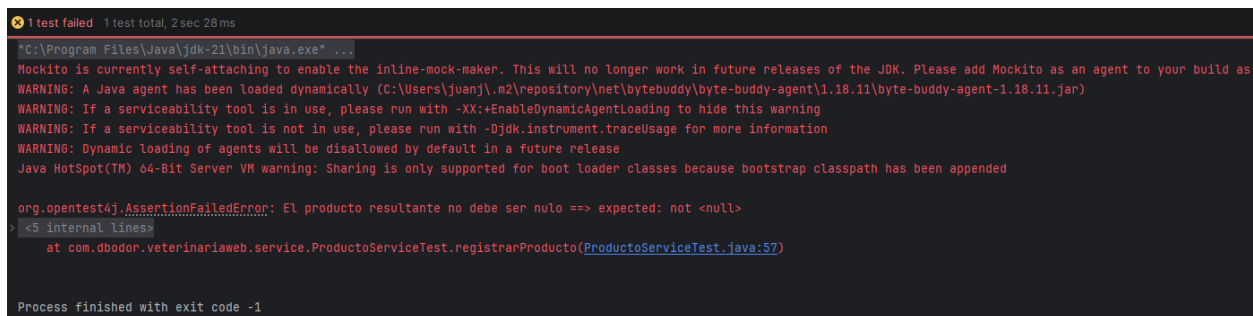
@Override
public Producto registrarProducto(Producto producto) {
    producto.setEstado(EstadoProducto.ACTIVO);
    return productoRepository.save(producto);
}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se ejecutó la prueba antes de codificar la lógica en el servicio (ProductoServiceImpl retornaba null). La aserción falló intencionalmente con AssertionError: El producto resultante no debe ser nulo ==> expected: not <null>.
- **Evidencia Visual:**



```

✖ 1 test failed 1 test total, 2 sec 28 ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

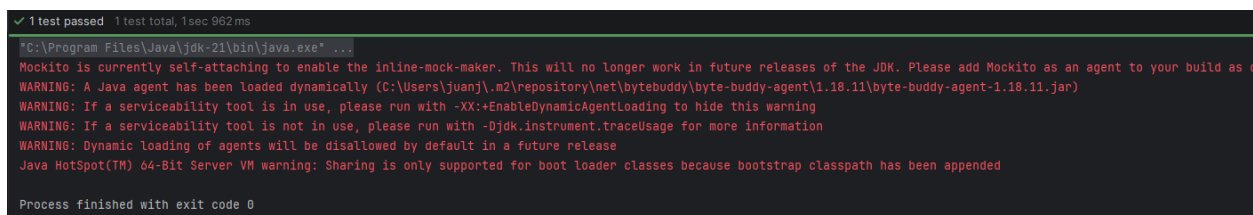
org.opentest4j.AssertionFailedError: El producto resultante no debe ser nulo ==> expected: not <null>
> <5 internal lines>
    at com.dbodor.veterinariaweb.service.ProductoServiceTest.registrarProducto(ProductoServiceTest.java:57)

Process finished with exit code -1

```

2. Fase Verde (GREEN):

- **Descripción:** Se asignó el estado EstadoProducto.ACTIVO y se delegó la persistencia mediante productoRepository.save(producto). La prueba pasó exitosamente en verde (1 of 1 passed).
- **Evidencia Visual:**



```

✓ 1 test passed 1 test total, 1 sec 962 ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0

```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se verificó la consistencia en el uso de enums tipados (EstadoProducto.ACTIVO) y la inyección limpia del repositorio por constructor.
-

Criterio de Aceptación 2:

Dado que el administrador ingresa un precio unitario menor o igual a cero o una cantidad de stock negativa, **cuando** intenta guardar el registro, **entonces** el sistema cancela la persistencia, resalta el campo numérico erróneo y muestra la alerta "El precio debe ser superior a cero y el stock no puede ser negativo".

A. Código

Código de la Prueba Unitaria (ProductoServiceTest.java):

@Test

```
@DisplayName("Debe lanzar IllegalArgumentException cuando el precio es <= 0")
```

```
void registrarProducto_precioInvalido() {  
    productoEntrada = new Producto();  
    productoEntrada.setPrecioUnitario(0.0);  
    productoEntrada.setStock(10);
```

```
    IllegalArgumentException exception = assertThrows(  
        IllegalArgumentException.class,  
        () -> productoService.registrarProducto(productoEntrada)  
    );
```

```
    assertEquals(  
        "El precio debe ser superior a cero y el stock no puede ser negativo",  
        exception.getMessage()  
    );  
}
```

@Test

```
@DisplayName("Debe lanzar IllegalArgumentException si el stock es < 0")
```

```
void registrarProducto_stockNegativo() {  
    productoEntrada = new Producto();  
    productoEntrada.setPrecioUnitario(45000.0);  
    productoEntrada.setStock(-1);
```

```
    IllegalArgumentException exception = assertThrows(  
        IllegalArgumentException.class,  
        () -> productoService.registrarProducto(productoEntrada)  
    );
```

```

    assertEquals(
        "El precio debe ser superior a cero y el stock no puede ser negativo",
        exception.getMessage()
    );
}

```

Código de Producción Implementado (ProductoServiceImpl.java):

```

@Override
public Producto registrarProducto(Producto producto) {
    if (producto.getPrecioUnitario() == null || producto.getPrecioUnitario() <= 0
        || producto.getStock() == null || producto.getStock() < 0) {
        throw new IllegalArgumentException("El precio debe ser superior a cero y el stock no puede ser negativo");
    }

    producto.setEstado(EstadoProducto.ACTIVO);
    return productoRepository.save(producto);
}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se ejecutó el test configurando un valor de precio inválido (0.0). Como el servicio no validaba rangos, la excepción esperada no fue lanzada y JUnit reportó: org.opentest4j.AssertionFailedError: Expected java.lang.IllegalArgumentException to be thrown, but nothing was thrown.
- **Evidencia Visual:**

```

1 test failed 1 test total, 2 sec 150 ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: Expected java.lang.IllegalArgumentException to be thrown, but nothing was thrown.
> <4 internal lines>
    at com.dbodor.veterinariaweb.service.ProductoServiceTest.registrarProducto_datosInvalidos_lanzaExcepcion(ProductoServiceTest.java:69)

Process finished with exit code -1

```

```

1 test failed 1 test total, 1 sec 993 ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: Expected java.lang.IllegalArgumentException to be thrown, but nothing was thrown.
> <4 internal lines>
    at com.dbodor.veterinariaweb.service.ProductoServiceTest.registrarProducto_stockNegativo(ProductoServiceTest.java:87)

Process finished with exit code -1

```

2. Fase Verde (GREEN):

- **Descripción:** Se agregó la guarda condicional en ProductoServiceImpl validando que precioUnitario > 0 y stock >= 0. Al cumplirse la condición de error, se cancela la ejecución lanzando IllegalArgumentException. Ambos tests pasaron en verde (2 of 2 passed).
- **Evidencia Visual:**

```
✓ 1 test passed 1 test total, 2 sec 116 ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.10.11\byte-buddy-agent-1.10.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se comprobó la protección contra valores nulos antes de comparar magnitudes y se verificó que el comportamiento del Criterio 1 sigue intacto.

Criterio de Aceptación 3:

Dado que un medicamento ya no se comercializa ni se utiliza en la clínica, **cuando** el administrador presiona el botón "Desactivar", **entonces** el sistema cambia su estado a INACTIVO, retirándose del catálogo, manteniendo intacto el registro en las historias clínicas pasadas.

A. Código

Código de la Prueba Unitaria (ProductoServiceTest.java):

```
@Test
```

```
@DisplayName("Criterio 3: Debe cambiar el estado del producto a INACTIVO al desactivarlo")
```

```
void desactivarProducto_cambiaEstadoAlInactivo() {
```

```
    Long idProducto = 1L;
```

```
    Producto productoExistente = new Producto();
```

```
    productoExistente.setIdProducto(idProducto);
```

```
    productoExistente.setNombre("NexGard Spectra");
```

```

    productoExistente.setEstado(EstadoProducto.ACTIVO);

    when(productoRepository.findById(idProducto)).thenReturn(java.util.Optional.of(productoExistente));

    when(productoRepository.save(any(Producto.class))).thenAnswer(invocation ->
    invocation.getArgument(0));

    Producto resultado = productoService.desactivarProducto(idProducto);

    assertNotNull(resultado, "El producto resultante no debe ser nulo");

    assertEquals(EstadoProducto.INACTIVO, resultado.getEstado(), "El estado del producto debe ser
    INACTIVO");

}

```

Código de Producción Implementado (ProductoServiceImpl.java):

```

@Override

public Producto desactivarProducto(Long idProducto) {

    Producto producto = productoRepository.findById(idProducto)

        .orElseThrow(() -> new IllegalArgumentException("No se encontró el producto con ID: " +
    idProducto));

    producto.setEstado(EstadoProducto.INACTIVO);

    return productoRepository.save(producto);

}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se ejecutó la prueba cuando el método desactivarProducto retornaba null. La aserción falló intencionalmente con `AssertionFailedError: El producto resultante no debe ser nulo ==> expected: not .`
- **Evidencia Visual:**

```
✖ 1 test failed 1 test total, 2 sec 124 ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: El producto resultante no debe ser nulo ==> expected: not <null>
> <5 internal lines>
    at com.dbodor.veterinariaweb.service.ProductoServiceTest.desactivarProducto_cambiaEstadoAInactivo(ProductoServiceTest.java:113)

Process finished with exit code -1
```

2. Fase Verde (GREEN):

- **Descripción:** Se implementó la búsqueda por ID con findById, la modificación de estado a EstadoProducto.INACTIVO y la persistencia mediante save en el repositorio. La prueba en JUnit pasó exitosamente en verde.
- **Evidencia Visual:**

```
✔ 1 test passed 1 test total, 2 sec 129 ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se aseguró la gestión de excepciones usando orElseThrow con mensaje descriptivo y se verificó que la firma mantuviera la consistencia de los datos históricos.

Criterio de Aceptación 4:

Dado que una cita programada tiene asociado el servicio "Consulta veterinaria", **cuando** el usuario finaliza la cita, **entonces** el sistema prescribe de forma predeterminada un antiparasitario en una notificación al usuario.

A. Código

Código de la Prueba Unitaria (CitaServiceTest.java):

@Test

```
@DisplayName("Criterio 4: Al finalizar cita de 'Consulta veterinaria' debe prescribir antiparasitario predeterminado")
```

```
void finalizarCita_prescribeAntiparasitarioPredeterminado() {
```

```
    Long idCita = 1L;
```

```
    Servicio servicioConsulta = new Servicio();
```

```
    servicioConsulta.setNombre("Consulta veterinaria");
```

```
    Cita cita = new Cita();
```

```
    cita.setIdCita(idCita);
```

```
    cita.setServicio(servicioConsulta);
```

```
    cita.setEstado("PROGRAMADA");
```

```
    Producto antiparasitario = new Producto();
```

```
    antiparasitario.setIdProducto(10L);
```

```
    antiparasitario.setNombre("Antiparasitario Canino/Felino");
```

```
    antiparasitario.setEstado(EstadoProducto.ACTIVO);
```

```
    antiparasitario.setStock(20);
```

```
    when(citaRepository.findById(idCita)).thenReturn(Optional.of(cita));
```

```
    when(productoRepository.findByNombreContainingIgnoreCase("Antiparasitario"))
```

```
        .thenReturn(Optional.of(antiparasitario));
```

```
    DetalleRecetaCita resultado = citaService.finalizarCita(idCita);
```

```
    assertNotNull(resultado, "El detalle de la receta prescrita no debe ser nulo");
```

```
    assertNotNull(resultado.getProducto(), "Debe contener el producto prescrito");
```

```
    assertEquals("El producto prescrito debe ser el antiparasitario predeterminado", "Antiparasitario Canino/Felino", resultado.getProducto().getNombre());
```

```
    Assertions.assertEquals(1, (int) resultado.getCantidad(), "La dosis sugerida por defecto debe ser 1");
```

```
    assertTrue(resultado.getEsSugerenciaCompra(), "Debe marcarse como sugerencia de compra / notificación");
```

```
    assertEquals("El estado de la cita debe actualizarse a FINALIZADA", "FINALIZADA",  
cita.getEstado());  
}
```

Código de Producción Implementado (CitaServiceImpl.java):

@Override

```
    public DetalleRecetaCita finalizarCita(Long idCita) {  
        Cita cita = citaRepository.findById(idCita)  
            .orElseThrow(() -> new IllegalArgumentException("No se encontró la cita con ID: " + idCita));  
  
        DetalleRecetaCita detallePrescripcion = null;  
  
        if (cita.getServicio() != null && "Consulta  
veterinaria".equalsIgnoreCase(cita.getServicio().getNombre())) {  
            Producto antiparasitario =  
productoRepository.findByNombreContainingIgnoreCase("Antiparasitario")  
                .orElseThrow(() -> new IllegalStateException("No se encontró un producto antiparasitario en  
el catálogo"));  
  
            detallePrescripcion = new DetalleRecetaCita();  
            detallePrescripcion.setProducto(antiparasitario);  
            detallePrescripcion.setCantidad(1);  
            detallePrescripcion.setPosologiaIndicaciones("Prescripción sugerida automáticamente tras  
consulta médica");  
            detallePrescripcion.setEsSugerenciaCompra(true);  
        }  
  
        cita.setEstado("FINALIZADA");  
        citaRepository.save(cita);  
    }
```

```
return detallePrescripcion;
```

```
}
```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se ejecutó el test cuando el método finalizarCita retornaba null. La aserción falló intencionalmente con AssertionError: El detalle de la receta prescrita no debe ser nulo ==> expected: not .
- **Evidencia Visual:**

```
✖ 1 test failed 1 test total, 2 sec 99 ms

C:\Program Files\Java\jdk-21\bin\java.exe ...

Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: El detalle de la receta prescrita no debe ser nulo ==> expected: not <null>
> <5 internal lines>
    at com.dbodor.veterinariaweb.service.CitaServiceTest.finalizarCita_prescribeAntiparasitarioPredeterminado(CitaServiceTest.java:60)

Process finished with exit code -1
```

2. Fase Verde (GREEN):

- **Descripción:** Se implementó la verificación del servicio "Consulta veterinaria", la consulta del antiparasitario mediante productoRepository y la asignación del objeto DetalleRecetaCita como sugerencia de compra con cantidad 1, marcando la cita en estado FINALIZADA. La prueba pasó a verde.
- **Evidencia Visual:**

```
✔ 1 test passed 1 test total, 2 sec 87 ms

C:\Program Files\Java\jdk-21\bin\java.exe ...

Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se aseguró la validación contra valores nulos en el servicio de la cita y la consistencia en el estado de la entidad antes de persistir.

Criterio de Aceptación 5:

Dado que una cita programada tiene asociado un servicio con prescripción predeterminada y el stock disponible de dicho producto en el catálogo es igual a 0, **cuando** el sistema intenta finalizar y procesar la cita en el sistema, **entonces** el sistema completa la cita médica con éxito, registra el medicamento en el historial clínico indicando la posología, marca el producto como *"Agotado temporalmente en farmacia"*, omite el enlace de compra directa en la notificación enviada al cliente y genera una alerta interna para que el administrador gestione el reabastecimiento del producto.

A. Código Fuente Involucrado

Código de la Prueba Unitaria (CitaServiceTest.java):

```
@Test
```

```
    @DisplayName("Criterio 5: Debe lanzar IllegalStateException si el producto a prescribir no tiene stock disponible")
```

```
    void finalizarCita_antiparasitarioSinStock_lanzaExcepcion() {  
        Long idCita = 2L;
```

```
        Servicio servicioConsulta = new Servicio();  
        servicioConsulta.setNombre("Consulta veterinaria");
```

```
        Cita cita = new Cita();  
        cita.setIdCita(idCita);  
        cita.setServicio(servicioConsulta);  
        cita.setEstado("PROGRAMADA");
```

```
        Producto antiparasitarioAgotado = new Producto();  
        antiparasitarioAgotado.setIdProducto(10L);  
        antiparasitarioAgotado.setNombre("Antiparasitario Canino/Felino");  
        antiparasitarioAgotado.setEstado(EstadoProducto.ACTIVO);  
        antiparasitarioAgotado.setStock(0);
```

```
        when(citaRepository.findById(idCita)).thenReturn(Optional.of(cita));  
        when(productoRepository.findByNombreContainingIgnoreCase("Antiparasitario"))  
            .thenReturn(Optional.of(antiparasitarioAgotado));
```

```
        IllegalStateException exception = assertThrows(  
            IllegalStateException.class,  
            () -> citaService.finalizarCita(idCita),  
            "Debe lanzar excepción si el producto a prescribir no cuenta con stock disponible"  
        );
```

```

        assertTrue(
            "El producto prescrito no cuenta con stock disponible".equals(exception.getMessage()),
            "El mensaje de error debe alertar sobre la disponibilidad de stock"
        );
    }
}

```

Código de Producción Implementado (CitaServiceImpl .java):

```

@Override
public DetalleRecetaCita finalizarCita(Long idCita) {
    Cita cita = citaRepository.findById(idCita)
        .orElseThrow(() -> new IllegalArgumentException("No se encontró la cita con ID: " + idCita));

    DetalleRecetaCita detallePrescripcion = null;

    if (cita.getServicio() != null && "Consulta
veterinaria".equalsIgnoreCase(cita.getServicio().getNombre())) {
        Producto antiparasitario =
        productoRepository.findByNombreContainingIgnoreCase("Antiparasitario")
            .orElseThrow(() -> new IllegalStateException("No se encontró un producto antiparasitario en
el catálogo"));

        if (antiparasitario.getStock() == null || antiparasitario.getStock() <= 0) {
            throw new IllegalStateException("El producto prescrito no cuenta con stock disponible");
        }

        detallePrescripcion = new DetalleRecetaCita();
        detallePrescripcion.setProducto(antiparasitario);
        detallePrescripcion.setCantidad(1);
        detallePrescripcion.setPosologiaIndicaciones("Prescripción sugerida automáticamente tras
consulta médica");
        detallePrescripcion.setEsSugerenciaCompra(true);
    }

    cita.setEstado("FINALIZADA");
    citaRepository.save(cita);

    return detallePrescripcion;
}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se ejecutó el test cuando el método finalizarCita no validaba el inventario del antiparasitario antes de armar la receta. La prueba falló intencionalmente con AssertionError: Expected java.lang.IllegalStateException to be thrown, but nothing was thrown.
- **Evidencia Visual:**

```
✖ 1 test failed 1 test total, 2sec 116ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: Debe lanzar excepción si el producto a prescribir no cuenta con stock disponible ==> Expected java.lang.IllegalStateException to be thrown but was null
> <4 internal lines>
    at com.dbodor.VeterinariaWeb.service.CitaServiceTest.finalizarCita_antiparasitarioSinStock_lanzaExcepcion(CitaServiceTest.java:91)

Process finished with exit code -1
```

2. Fase Verde (GREEN):

- **Descripción:** Se agregó la condición de guarda que valida si `antiparasitario.getStock()` es nulo o menor o igual a cero, arrojando `IllegalStateException` antes de asignar el detalle o guardar la cita. Ambas aserciones pasaron a verde.
- **Evidencia Visual:**

```
✔ 1 test passed 1 test total, 2sec 91ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se validó la protección integral de nulidad sobre el campo `stock` y se garantizó la atomicidad transaccional impidiendo que la entidad `Cita` cambie a estado `FINALIZADA` si la prescripción predeterminada no puede abastecerse.

Épica: Gestión de Servicios Veterinarios

Historia de Usuario HU-19: Consultar el catálogo de servicios

- **Como:** Cliente
- **Quiero:** Ver los servicios disponibles con su precio y duración
- **Para:** Decidir cuál agendar.

Criterio de Aceptación 1:

Dado que ingreso como cliente, **cuando** abro el catálogo, **entonces** solo veo los servicios en estado activo.

A. Código

Código de la Prueba Unitaria (HU19ConsultarServiciosTest.java):

```
@Test

@DisplayName("Criterio 1: Cliente solo debe visualizar servicios en estado activo")

void listarServiciosParaCliente_retornaSoloServiciosActivos() {

    Servicio s1 = new Servicio();

    s1.setIdServicio(1L);

    s1.setNombre("Consulta general");

    s1.setEstado(EstadoServicio.ACTIVO);


    Servicio s2 = new Servicio();

    s2.setIdServicio(2L);

    s2.setNombre("Vacunación antirrábica");

    s2.setEstado(EstadoServicio.ACTIVO);


    when(servicioRepository.findByEstado(EstadoServicio.ACTIVO)).thenReturn(List.of(s1, s2));


    List<Servicio> resultado = servicioService.listarServiciosParaCliente();


    assertNotNull(resultado, "La lista de servicios no debe ser nula");

    assertEquals(2, resultado.size(), "Debe retornar únicamente los 2 servicios activos");

    assertTrue(resultado.stream().allMatch(s -> s.getEstado() == EstadoServicio.ACTIVO),
        "Todos los servicios listados deben estar activos");

}
```

Código de Producción Implementado (ServicioServiceImpl.java):

```
@Override
```

```

public List<Servicio> listarServiciosParaCliente() {

    return servicioRepository.findByEstado(EstadoServicio.ACTIVO);

}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se ejecutó el test cuando el método listarServiciosParaCliente retornaba null. La prueba falló intencionalmente con AssertionError: La lista de servicios no debe ser nula ==> expected: not .
- **Evidencia Visual:**

```

✖ 1 test failed 1 test total, 2 sec 82ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: La lista de servicios no debe ser nula ==> expected: not <null>
> <5 internal lines>
    at com.dboron.veterinariaweb.service.HU19ConsultarServiciosTest.listarServiciosParaCliente_retornaSoloServiciosActivos(HU19ConsultarServiciosTest.java:46)

Process finished with exit code -1

```

2. Fase Verde (GREEN):

- **Descripción:** Se implementó la consulta en el repositorio mediante findByEstado(EstadoServicio.ACTIVO), retornando únicamente las entidades con estado ACTIVO. La prueba en JUnit pasó a verde.
- **Evidencia Visual:**

```

✔ 1 test passed 1 test total, 2 sec 55ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0

```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se estructuró el tipo enumerado EstadoServicio y se aseguró el desacoplamiento entre las consultas públicas de clientes y las consultas globales administrativas.

Criterio de Aceptación 2:

Dado que consulto el catálogo, **cuando** lo reviso, **entonces** cada servicio muestra nombre, descripción, duración y precio.

A. Código

Código de la Prueba Unitaria (HU19ConsultarServiciosTest.java):

@Test

@DisplayName("Criterio 2: Cada servicio del catálogo debe contener nombre, descripción, duración y precio")

```
void listarServiciosParaCliente_validaCamposObligatorios() {
```

```
    Servicio servicioCompleto = new Servicio();
```

```
    servicioCompleto.setIdServicio(1L);
```

```
    servicioCompleto.setNombre("Consulta general");
```

```
    servicioCompleto.setDescripcion("Evaluación física completa del paciente");
```

```
    servicioCompleto.setDuracionMinutos(30);
```

```
    servicioCompleto.setPrecioBase(45000.00);
```

```
    servicioCompleto.setEstado(EstadoServicio.ACTIVO);
```

```
    when(servicioRepository.findByEstado(EstadoServicio.ACTIVO))
```

```
        .thenReturn(List.of(servicioCompleto));
```

```
    List<Servicio> resultado = servicioService.listarServiciosParaCliente();
```

```
    assertNotNull(resultado, "La lista no debe ser nula");
```

```
    assertFalse(resultado.isEmpty(), "La lista debe contener elementos");
```

```

Servicio servicio = resultado.getFirst();

assertNotNull(servicio.getNombre(), "El nombre del servicio no debe ser nulo");

assertFalse(servicio.getNombre().isBlank(), "El nombre del servicio no debe estar vacío");


assertNotNull(servicio.getDescripcion(), "La descripción no debe ser nula");

assertFalse(servicio.getDescripcion().isBlank(), "La descripción no debe estar vacía");


assertNotNull(servicio.getDuracionMinutos(), "La duración no debe ser nula");

assertTrue(servicio.getDuracionMinutos() > 0, "La duración debe ser mayor a cero minutos");


assertNotNull(servicio.getPrecioBase(), "El precio no debe ser nulo");

assertTrue(servicio.getPrecioBase() > 0, "El precio debe ser mayor a cero");

}

```

Código de Producción Implementado (ServicioServiceImpl.java):

```

@Override

public List<Servicio> listarServiciosParaCliente() {

    return servicioRepository.findByEstado(EstadoServicio.ACTIVO);

}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se configuró el método listarServiciosParaCliente para retornar una colección vacía (List.of()). La aserción falló intencionalmente con AssertionError: La lista debe contener elementos.
- **Evidencia Visual:**

```
✖ 1 test failed 1 test total, 1sec 987ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: La lista debe contener elementos ==>
Expected :false
Actual   :true
<Click to see difference>

> <5 internal lines>
    at com.dbodor.veterinariaweb.service.HU19ConsultarServiciosTest.listarServiciosParaCliente_validaCamposObligatorios(HU19ConsultarServiciosTest.java:68)

Process finished with exit code -1
```

2. Fase Verde (GREEN):

- **Descripción:** Se restauró la invocación a `servicioRepository.findByEstado(EstadoServicio.ACTIVO)`, permitiendo recuperar los objetos completos con todos sus atributos requeridos (nombre, descripción, duración en minutos y precio base). La suite pasó a verde.
- **Evidencia Visual:**

```
✔ 1 test passed 1 test total, 1sec 992ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se verificó la consistencia en el uso del tipo primitivo envoltorio `Double` para el atributo `precioBase` y la validación de campos de texto mediante `isBlank()` para evitar cadenas con solo espacios en blanco.

Criterio de Aceptación 3:

Dado que soy administrador, **cuando** abro el mismo maestro, **entonces** veo también los servicios inactivos marcados como tales.

A. Código

@Test

@DisplayName("Criterio 3: Administrador debe visualizar tanto servicios activos como inactivos en el maestro")

void listarTodosLosServicios_retornaActivosEInactivosParaAdmin() {

```

    Servicio sActivo = new Servicio();
    sActivo.setIdServicio(1L);
    sActivo.setNombre("Consulta general");
    sActivo.setEstado(EstadoServicio.ACTIVO);

    Servicio sInactivo = new Servicio();
    sInactivo.setIdServicio(2L);
    sInactivo.setNombre("Peluquería canina avanzada");
    sInactivo.setEstado(EstadoServicio.INACTIVO);

    when(servicioRepository.findAll()).thenReturn(List.of(sActivo, sInactivo));

    List<Servicio> resultado = servicioService.listarTodosLosServicios();

    assertNotNull(resultado, "La lista de servicios para admin no debe ser nula");
    assertTrue(resultado.size() == 2, "Debe contener el total de servicios sin filtrar");

    boolean contieneActivo = resultado.stream().anyMatch(s -> s.getEstado() ==
EstadoServicio.ACTIVO);
    boolean contieneInactivo = resultado.stream().anyMatch(s -> s.getEstado() ==
EstadoServicio.INACTIVO);

    assertTrue(contieneActivo, "El maestro debe incluir los servicios en estado ACTIVO");
    assertTrue(contieneInactivo, "El maestro debe incluir los servicios en estado INACTIVO marcados
como tales");
}

```

Código de Producción Implementado (ServicioServiceImpl.java):

```

@Override
public List<Servicio> listarTodosLosServicios() {
    return servicioRepository.findAll();
}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se ejecutó el test con el método listarTodosLosServicios retornando null. La aserción falló intencionalmente con AssertionError: La lista de servicios para admin no debe ser nula ==> expected: not .
- **Evidencia Visual:**

```
✖ 1 test failed 1 test total, 2 sec 39 ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...

Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juan\m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: La lista de servicios para admin no debe ser nula ==> expected: not <null>
> <5 internal lines>
    at com.dbodor.veterinariaweb.service.HU19ConsultarServiciosTest.listarTodosLosServicios_retornaActivosEInactivosParaAdmin(HU19ConsultarServiciosTest.java:101)

Process finished with exit code -1
```

2. Fase Verde (GREEN):

- **Descripción:** Se implementó la invocación a `findAll()` en el repositorio para retornar la totalidad de los registros sin discriminación de estado, permitiendo que el rol administrador audite tanto servicios en estado ACTIVO como INACTIVO con sus respectivos distintivos. La prueba pasó a verde.
- **Evidencia Visual:**

```
✔ 1 test passed 1 test total, 2 sec 44 ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...

Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juan\m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se garantizó la separación clara de responsabilidades entre la consulta pública de clientes (restringida a servicios activos) y la vista maestra del administrador mediante métodos independientes en la capa de servicio.

Historia de Usuario HU-20: Validar la duración del servicio

- **Como:** Administrador
- **Quiero:** Que la duración se registre en bloques válidos
- **Para:** Que la agenda no genere horarios imposibles de asignar.

Criterio de Aceptación 1:

Dado que registro la duración de un servicio, **cuando** el valor no es múltiplo de quince minutos, **entonces** el sistema lo rechaza.

A. Código

Código de la Prueba Unitaria (HU20ValidarDuracionServicioTest.java):

```
@Test
@DisplayName("Criterio 1: Debe rechazar el servicio si la duración no es múltiplo de 15 minutos")
void guardarServicio_duracionNoMultiploDe15_lanzaExcepcion() {
    Servicio servicioInvalido = new Servicio();
    servicioInvalido.setNombre("Corte de uñas express");
    servicioInvalido.setDescripcion("Corte rápido de uñas");
    servicioInvalido.setDuracionMinutos(25); // No es múltiplo de 15
    servicioInvalido.setPrecioBase(20000.0);
    servicioInvalido.setEstado(EstadoServicio.ACTIVO);

    IllegalArgumentException excepcion = assertThrows(
        IllegalArgumentException.class,
        () -> servicioService.guardarServicio(servicioInvalido),
        "Debe lanzar IllegalArgumentException cuando la duración no es múltiplo de 15"
    );
    assertTrue(
        "La duración debe ser un múltiplo de 15 minutos".equals(excepcion.getMessage()),
        "El mensaje debe indicar que la duración debe ser múltiplo de 15 minutos"
    );
}
```

Código de Producción Implementado (ServicioServiceImpl.java):

```
@Override
public Servicio guardarServicio(Servicio servicio) {
    if (servicio.getDuracionMinutos() == null || servicio.getDuracionMinutos() % 15 != 0) {
        throw new IllegalArgumentException("La duración debe ser un múltiplo de 15 minutos");
    }
    return servicioRepository.save(servicio);
}
```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se ejecutó el test cuando el método guardarServicio no implementaba reglas de validación sobre el campo duracionMinutos (retornaba null o no arrojaba excepción). La prueba falló intencionalmente con AssertionError: Expected java.lang.IllegalArgumentException to be thrown, but nothing was thrown.
- **Evidencia Visual:**

```

✖ 1 test failed 1 test total, 2 sec 329ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build a
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: Debe lanzar IllegalArgumentException cuando la duración no es múltiplo de 15 ==> Expected java.lang.IllegalArgumentException to be thr
> <4 internal lines>
    at com.dbodor.veterinariaweb.service.HU20ValidarDuracionServicioTest.guardarServicio_duracionNoMultiploDe15_lanzaExcepcion(HU20ValidarDuracionServicioTest.java:37)

Process finished with exit code -1

```

2. Fase Verde (GREEN):

- **Descripción:** Se agregó la condición de guarda evaluando la operación módulo (% 15 != 0) y comprobando la no nulidad, arrojando IllegalArgumentException con el mensaje correspondiente antes de permitir la persistencia en el repositorio. La prueba en JUnit pasó a verde.
- **Evidencia Visual:**

```

✔ 1 test passed 1 test total, 2 sec 11ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as d
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0

```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se aseguró la validación previa de nulidad en el atributo Integer para prevenir NullPointerException en tiempo de ejecución antes de evaluar la operación aritmética del módulo.

Criterio de Aceptación 2:

Dado que la duración es menor a quince minutos o mayor a ciento ochenta, **cuando** guardo, **entonces** el sistema rechaza el valor y explica el rango permitido.

A. Código

@Test

```
@DisplayName("Criterio 2: Debe rechazar el servicio si la duración es menor a 15 minutos y explicar el rango")
```

```
void guardarServicio_duracionMenorA15_lanzaExcepcionConMensajeExplicativo() {
```

```
    Servicio servicioMenor = new Servicio();
```

```
    servicioMenor.setNombre("Revisión");
```

```
    servicioMenor.setDescripcion("Revisión");
```

```
    servicioMenor.setDuracionMinutos(0);
```

```
    servicioMenor.setPrecioBase(40000.0);
```

```
    servicioMenor.setEstado(EstadoServicio.ACTIVO);
```

```
    IllegalArgumentException excepcion = assertThrows(
```

```
        IllegalArgumentException.class,
```

```
        () -> servicioService.guardarServicio(servicioMenor),
```

```
        "Debe lanzar excepción si la duración es menor a 15 minutos"
```

```
    );
```

```
    assertTrue(
```

```
        "La duración debe estar entre 15 y 180 minutos".equals(excepcion.getMessage()),
```

```
        "El mensaje debe explicar el rango permitido de 15 a 180 minutos"
```

```
    );
```

```
}
```

```
@Test
```

```
@DisplayName("Criterio 2: Debe rechazar el servicio si la duración es mayor a 180 minutos y explicar el rango")
```

```
void guardarServicio_duracionMayorA180_lanzaExcepcionConMensajeExplicativo() {
```

```
    Servicio servicioMayor = new Servicio();
```

```
    servicioMayor.setNombre("Cirugía compleja reconstructiva");
```

```
    servicioMayor.setDescripcion("Procedimiento quirúrgico de alta complejidad");
```

```
    servicioMayor.setDuracionMinutos(195);
```

```
    servicioMayor.setPrecioBase(250000.0);
```

```
    servicioMayor.setEstado(EstadoServicio.ACTIVO);
```

```
    IllegalArgumentException excepcion = assertThrows(
```

```
        IllegalArgumentException.class,
```

```
        () -> servicioService.guardarServicio(servicioMayor),
```

```
        "Debe lanzar excepción si la duración supera los 180 minutos"
```

```
    );
```

```
    assertTrue(
```

```
        "La duración debe estar entre 15 y 180 minutos".equals(excepcion.getMessage()),
```

```
        "El mensaje debe explicar el rango permitido de 15 a 180 minutos"
```

```
    );
```

```
}
```

Código de Producción Implementado (ServicioServiceImpl.java):

```
@Override
```

```

public Servicio guardarServicio(Servicio servicio) {
    if (servicio.getDuracionMinutos() == null) {
        throw new IllegalArgumentException("La duración no puede ser nula");
    }

    if (servicio.getDuracionMinutos() < 15 || servicio.getDuracionMinutos() > 180) {
        throw new IllegalArgumentException("La duración debe estar entre 15 y 180 minutos");
    }

    if (servicio.getDuracionMinutos() % 15 != 0) {
        throw new IllegalArgumentException("La duración debe ser un múltiplo de 15 minutos");
    }

    return servicioRepository.save(servicio);
}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se ejecutaron los tests parametrizando duraciones fuera de los límites (0 y 195 minutos). Al solo comprobarse previamente la divisibilidad aritmética ($\% 15 \neq 0$), el servicio permitió el valor de 195 minutos sin lanzar excepción, fallando con `AssertionFailedError: Expected java.lang.IllegalArgumentException to be thrown, but nothing was thrown.`
- **Evidencia Visual:**

```

✖ 1 test failed 1 test total, 2 sec 147ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Ojdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: Debe lanzar excepción si la duración es menor a 15 minutos ==> Expected java.lang.IllegalArgumentException to be thrown, but nothing w
> <4 internal lines>
    at com.dbodorb.veterinariaweb.service.HU20ValidarDuracionServicioTest.guardarServicio_duracionMenorA15_lanzaExcepcionConMensajeExplicativo(HU20ValidarDuracionServicioTe
Process finished with exit code -1

```

```

✖ 1 test failed 1 test total, 2 sec 185ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Ojdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: Debe lanzar excepción si la duración supera los 180 minutos ==> Expected java.lang.IllegalArgumentException to be thrown, but nothing w
> <4 internal lines>
    at com.dbodorb.veterinariaweb.service.HU20ValidarDuracionServicioTest.guardarServicio_duracionMayorA180_lanzaExcepcionConMensajeExplicativo(HU20ValidarDuracionServicioTe
Process finished with exit code -1

```

2. Fase Verde (GREEN):

- **Descripción:** Se incorporó la validación condicional del rango acotado entre 15 y 180 minutos con el mensaje de retroalimentación correspondiente, deteniendo la persistencia ante valores inválidos. La suite de pruebas en JUnit pasó a verde.
- **Evidencia Visual:**

```
✓ 1 test passed 1 test total, 2 sec 192 ms
C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0
```

```
✓ 1 test passed 1 test total, 2 sec 24 ms
C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se ordenaron las validaciones jerárquicamente: primero la verificación de no nulidad, posteriormente el rango absoluto (15-180) y finalmente la granularidad por múltiplos de 15 minutos, evitando cálculos innecesarios.

Criterio de Aceptación 3:

Dado que un servicio dura sesenta minutos, **cuando** se agenda a las 09:00, **entonces** el sistema bloquea la franja hasta las 10:00 para ese veterinario.

A. Código

@Test

```
@DisplayName("Criterio 3: Servicio de 60 min a las 09:00 bloquea la franja hasta las 10:00 impidiendo solapamientos")
```

```
void agendarCita_servicio60Minutos_bloqueaFranjaHastaLas10() {
    Veterinario vet = new Veterinario();
    vet.setIdVeterinario(1L);
```

```
    Servicio servicioExistente = new Servicio();
    servicioExistente.setIdServicio(1L);
```

```

servicioExistente.setNombre("Consulta general");
servicioExistente.setDuracionMinutos(60); // 60 minutos de duración

LocalDate fecha = LocalDate.of(2026, 10, 15);

// Cita existente de 09:00 a 10:00
Cita citaExistente = new Cita();
citaExistente.setIdCita(100L);
citaExistente.setVeterinario(vet);
citaExistente.setServicio(servicioExistente);
citaExistente.setFechaCita(fecha);
citaExistente.setHoraCita(LocalTime.of(9, 0)); // 09:00
citaExistente.setEstado("PROGRAMADA");

// Intento de nueva cita a las 09:30 (dentro de la franja ocupada 09:00 a 10:00)
Servicio servicioNuevo = new Servicio();
servicioNuevo.setIdServicio(2L);
servicioNuevo.setNombre("Vacunación");
servicioNuevo.setDuracionMinutos(30);

Cita nuevaCita = new Cita();
nuevaCita.setVeterinario(vet);
nuevaCita.setServicio(servicioNuevo);
nuevaCita.setFechaCita(fecha);
nuevaCita.setHoraCita(LocalTime.of(9, 30)); // 09:30

when(citaRepository.findByVeterinarioAndFechaCitaAndEstadoNot(vet, fecha, "CANCELADA"))
    .thenReturn(List.of(citaExistente));

IllegalStateException excepcion = assertThrows(
    IllegalStateException.class,
    () -> citaService.agendarCita(nuevaCita),
    "Debe lanzar excepción porque la franja está ocupada hasta las 10:00"
);

assertTrue(
    "El veterinario ya tiene una cita asignada en ese rango horario".equals(excepcion.getMessage()),
    "El mensaje debe indicar conflicto por franja horaria ocupada"
);
}

```

Código de Producción Implementado (CitaServiceImpl.java):

```

@Override
public Cita agendarCita(Cita cita) {
    if (cita.getServicio() == null || cita.getServicio().getDuracionMinutos() == null) {
        throw new IllegalArgumentException("La cita debe tener un servicio con duración válida");
    }
}

```

```

        if (cita.getCostoTotal() == null) {
            cita.setCostoTotal(cita.getServicio().getPrecioBase());
        }

        LocalTime inicioNueva = cita.getHoraCita();
        LocalTime finNueva = inicioNueva.plusMinutes(cita.getServicio().getDuracionMinutos());

        List<Cita> citasExistentes = citaRepository.findByVeterinarioAndFechaCitaAndEstadoNot(
            cita.getVeterinario(),
            cita.getFechaCita(),
            "CANCELADA"
        );

        for (Cita existente : citasExistentes) {
            LocalTime inicioExistente = existente.getHoraCita();
            LocalTime finExistente =
                inicioExistente.plusMinutes(existente.getServicio().getDuracionMinutos());

            if (inicioNueva.isBefore(finExistente) && finNueva.isAfter(inicioExistente)) {
                throw new IllegalStateException("El veterinario ya tiene una cita asignada en ese rango
                horario");
            }
        }
        return citaRepository.save(cita);
    }

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se ejecutó la prueba con el método `agendarCita` retornando null sin aplicar la lógica de verificación de solapamientos ni cálculo de horas de finalización de franja. La prueba falló intencionalmente con `AssertionFailedError: Expected java.lang.IllegalStateException to be thrown, but nothing was thrown.`
- **Evidencia Visual:**

```

❌ 1 test failed 1 test total, 2 sec 321ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: Debe lanzar excepción porque la franja está ocupada hasta las 10:00 ==> Expected java.lang.IllegalStateException to be thrown, but nothing
> <4 internal lines>
    at com.dbodor.veterinariaweb.service.HU20ValidarDuracionServicioTest.agendarCita_servicio60Minutos_bloqueaFranjaHastaLas10(HU20ValidarDuracionServicioTest.java:144)

Process finished with exit code -1

```

2. Fase Verde (GREEN):

- **Descripción:** Se implementó la lógica de cálculo dinámico de la franja horaria mediante plusMinutes() según la duración del servicio asociado y la evaluación de colisión temporal entre intervalos [inicio, fin), rechazando citas solapadas con IllegalStateException. La suite en JUnit pasó a verde.
- **Evidencia Visual:**

```
✓ 1 test passed 1 test total, 2 sec 245 ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se aplicó el algoritmo estándar de intersección de rangos temporales semiabiertos (inicioA < finB y finA > inicioB), filtrando previamente las citas en estado CANCELADA para evitar bloqueos injustificados en la agenda del profesional.

Historia de Usuario HU-17: Modificar el precio de un servicio

- **Como:** Administrador
- **Quiero:** Actualizar el precio de un servicio
- **Para:** Ajustarlo a la tarifa vigente.

Criterio de Aceptación 1:

Dado que edito un servicio, **cuando** cambio el precio y guardo, **entonces** el nuevo valor aplica solo a las citas agendadas a partir de ese momento.

A. Código

@Test

```
@DisplayName("Criterio 1: Modificar el precio de un servicio aplica el nuevo valor a citas posteriores")
```

```
void actualizarPrecio_aplicaSoloANuevasCitasAgendadas() {
```

```
    Servicio servicio = new Servicio();
```

```
servicio.setIdServicio(1L);  
servicio.setNombre("Consulta general");  
servicio.setDuracionMinutos(30);  
servicio.setPrecioBase(50000.0);  
servicio.setEstado(EstadoServicio.ACTIVO);
```

```
when(servicioRepository.findById(1L)).thenReturn(Optional.of(servicio));  
when(servicioRepository.save(any(Servicio.class))).thenReturn(invocation ->  
invocation.getArgument(0));
```

```
Servicio servicioActualizado = servicioService.actualizarPrecio(1L, 60000.0);
```

```
assertNotNull(servicioActualizado, "El servicio actualizado no debe ser nulo");  
assertEquals(60000.0, servicioActualizado.getPrecioBase(), "El nuevo precio base debe ser  
60000.0");
```

```
Veterinario vet = new Veterinario();  
vet.setIdVeterinario(1L);
```

```
Cita nuevaCita = new Cita();  
nuevaCita.setVeterinario(vet);  
nuevaCita.setServicio(servicioActualizado);  
nuevaCita.setFechaCita(LocalDate.of(2026, 11, 1));  
nuevaCita.setHoraCita(LocalTime.of(10, 0));
```

```
when(citaRepository.findByVeterinarioAndFechaCitaAndEstadoNot(any(), any(), any()))  
.thenReturn(List.of());  
when(citaRepository.save(any(Cita.class))).thenReturn(invocation -> invocation.getArgument(0));
```

```
Cita citaAgendada = citaService.agendarCita(nuevaCita);
```

```
assertNotNull(citaAgendada.getCostoTotal(), "El costo total de la nueva cita no debe ser nulo");
```

```

    assertEquals(60000.0, citaAgendada.getCostoTotal(),
        "La nueva cita debe registrarse con la tarifa vigente de 60000.0");
}

```

Código de Producción Implementado (ServicioServiceImpl.java y CitaServiceImpl.java):

```

@Override
public Servicio actualizarPrecio(Long idServicio, Double nuevoPrecio) {
    if (nuevoPrecio == null || nuevoPrecio <= 0.0) {
        throw new IllegalArgumentException("El precio debe ser un valor positivo");
    }

    Servicio servicio = servicioRepository.findById(idServicio)
        .orElseThrow(() -> new IllegalArgumentException("Servicio no encontrado con id: " +
idServicio));

    servicio.setPrecioBase(nuevoPrecio);
    return servicioRepository.save(servicio);
}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se configuró la prueba unitaria cuando el método actualizarPrecio retornaba un valor nulo por defecto en el servicio. La aserción falló de forma esperada con AssertionError: El servicio actualizado no debe ser nulo ==> expected: not .
- **Evidencia Visual:**

```

X 1 test failed 1 test total, 2 sec 72 ms
C:\Program Files\Java\jdk-21\bin\java.exe
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:-EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: El servicio actualizado no debe ser nulo ==> expected: not <null>
> <5 internal lines>
at com.dbodor.veterinariaweb.service.HU17ModificarPrecioServicioTest.actualizarPrecio_aplicaSoloANuevasCitasAgendadas(HU17ModificarPrecioServicioTest.java:58)

Process finished with exit code -1

```

2. Fase Verde (GREEN):

- **Descripción:** Se implementó la recuperación del servicio por ID, validación de positividad del valor y persistencia del nuevo precioBase en ServicioServiceImpl, garantizando además que agendarCita congele

dicho valor en el atributo costoTotal de las nuevas reservas. La prueba pasó a verde.

- **Evidencia Visual:**

```
✓ 1 test passed 1 test total, 2 sec 63ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as d
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se separó la mutabilidad del catálogo respecto al ciclo de vida de la transacción económica, garantizando que el precio del servicio opere como tarifa por defecto al agendar sin acoplar dinámicamente las citas pasadas.

Criterio de Aceptación 2:

Dado que existen citas ya atendidas con ese servicio, **cuando** cambio el precio, **entonces** el valor registrado en esas citas no se altera y los reportes históricos no cambian.

A. Código

@Test

@DisplayName("Criterio 2: Modificar el precio del servicio no altera el costoTotal de citas ya atendidas")

```
void actualizarPrecio_citasAtendidasConservanPrecioHistorico() {
```

```
    Servicio servicio = new Servicio();
```

```
    servicio.setIdServicio(1L);
```

```
    servicio.setNombre("Desparasitación");
```

```
    servicio.setDuracionMinutos(15);
```

```
    servicio.setPrecioBase(40000.0);
```

```
    servicio.setEstado(EstadoServicio.ACTIVO);
```

```
    Cita citaAtendida = new Cita();
```

```
    citaAtendida.setIdCita(50L);
```

```
    citaAtendida.setServicio(servicio);
```

```
    citaAtendida.setFechaCita(LocalDate.of(2026, 8, 10));
```

```
    citaAtendida.setHoraCita(LocalTime.of(14, 0));
```

```
    citaAtendida.setEstado("ATENDIDA");
```

```
    citaAtendida.setCostoTotal(40000.0);
```

```

        when(servicioRepository.findById(1L)).thenReturn(Optional.of(servicio));
        when(servicioRepository.save(any(Servicio.class))).thenAnswer(invocation ->
invocation.getArgument(0));

        Servicio servicioModificado = servicioService.actualizarPrecio(1L, 55000.0);

        assertEquals(55000.0, servicioModificado.getPrecioBase(), "El catálogo de servicios debe reflejar
55000.0");
        assertEquals(40000.0, citaAtendida.getCostoTotal(),
            "El costoTotal registrado en la cita atendida debe permanecer inalterado en 40000.0");
        assertNotEquals(servicioModificado.getPrecioBase(), citaAtendida.getCostoTotal(),
            "El costo histórico de la cita no debe igualarse al nuevo precio de catálogo");
    }

```

Código de Producción Implementado (ServicioServiceImpl.java):

```

@Override
public Servicio actualizarPrecio(Long idServicio, Double nuevoPrecio) {
    if (nuevoPrecio == null || nuevoPrecio <= 0.0) {
        throw new IllegalArgumentException("El precio debe ser un valor positivo");
    }

    Servicio servicio = servicioRepository.findById(idServicio)
        .orElseThrow(() -> new IllegalArgumentException("Servicio no encontrado con id: " +
idServicio));

    servicio.setPrecioBase(nuevoPrecio);
    return servicioRepository.save(servicio);
}

```

Historia de Usuario HU-18: Desactivar un servicio

- **Como:** Administrador
- **Quiero:** Desactivar un servicio que ya no se presta
- **Para:** Retirarlo de la oferta sin perder su historial.

Criterio de Aceptación 1:

Dado que un servicio está activo, **cuando** lo desactivo, **entonces** deja de aparecer en la pantalla de reservas.

A. Código

```

@Test
@DisplayName("Criterio 1: Desactivar un servicio lo excluye de la pantalla de reservas (cliente)")
void desactivarServicio_servicioActivo_dejaDeAparecerEnReservas() {

```

```

    Servicio servicio = new Servicio();
    servicio.setIdServicio(1L);
    servicio.setNombre("Peluquería Canina");
    servicio.setDuracionMinutos(45);
    servicio.setPrecioBase(35000.0);
    servicio.setEstado(EstadoServicio.ACTIVO);

    when(servicioRepository.findById(1L)).thenReturn(Optional.of(servicio));
    when(servicioRepository.save(any(Servicio.class))).thenReturn(invocation ->
invocation.getArgument(0));

    Servicio servicioDesactivado = servicioService.desactivarServicio(1L);

    assertNotNull(servicioDesactivado, "El servicio retornado no debe ser nulo");
    assertEquals(EstadoServicio.INACTIVO, servicioDesactivado.getEstado(),
        "El servicio debe quedar con estado INACTIVO");

    when(servicioRepository.findByEstado(EstadoServicio.ACTIVO)).thenReturn(List.of());

    List<Servicio> serviciosParaCliente = servicioService.listarServiciosParaCliente();

    assertTrue(serviciosParaCliente.isEmpty(),
        "La lista para cliente no debe incluir servicios inactivos");
    assertFalse(serviciosParaCliente.contains(servicioDesactivado),
        "El servicio desactivado no debe figurar en la pantalla de reservas");
}

```

Código de Producción Implementado (ServicioServiceImpl.java):

```

@Override
public Servicio desactivarServicio(Long idServicio) {
    Servicio servicio = servicioRepository.findById(idServicio)
        .orElseThrow(() -> new IllegalArgumentException("Servicio no encontrado con id: " +
idServicio));

    servicio.setEstado(EstadoServicio.INACTIVO);
    return servicioRepository.save(servicio);
}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se estructuró la prueba unitaria cuando el método desactivarServicio retornaba null por defecto en su firma inicial. La ejecución falló intencionalmente con AssertionError: El servicio retornado no debe ser nulo ==> expected: not .
- **Evidencia Visual:**

```
✖ 1 test failed 1 test total, 2 sec 113 ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: El servicio retornado no debe ser nulo ==> expected: not <null>
> <5 internal lines>
    at com.dbodor.veterinariaweb.service.HU18DesactivarServicioTest.desactivarServicio_servicioActivo_dejaDeAparecerEnReservas(HU18DesactivarServicioTest.java:52)

Process finished with exit code -1
```

2. Fase Verde (GREEN):

- **Descripción:** Se implementó la recuperación del servicio por su identificador primario, la transición de estado hacia INACTIVO y su persistencia mediante save(). Asimismo, se verificó que la consulta de catálogo para clientes filtre estrictamente por EstadoServicio.ACTIVO, ocultando los registros deshabilitados. La prueba pasó a verde.
- **Evidencia Visual:**

```
✔ 1 test passed 1 test total, 2 sec 77 ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se aplicó el patrón de borrado lógico (soft delete) a través del enum de estado, evitando la eliminación física (DELETE) en la base de datos para preservar la integridad referencial con el histórico de citas y clientes.

Criterio de Aceptación 2: Advertencia y Confirmación por Citas Pendientes

Dado que existen citas pendientes con ese servicio, **cuando** intento desactivarlo, **entonces** el sistema advierte y solicita confirmación explícita.

A. Código

@Test

@DisplayName("Criterio 2: Desactivar servicio con citas pendientes sin confirmación lanza advertencia")

```
void desactivarServicio_conCitasPendientesSinConfirmacion_lanzaAdvertencia() {
    Servicio servicio = new Servicio();
```

```

servicio.setIdServicio(1L);
servicio.setNombre("Vacunación");
servicio.setEstado(EstadoServicio.ACTIVO);

when(servicioRepository.findById(1L)).thenReturn(Optional.of(servicio));
// Simula que existen citas pendientes en estado PROGRAMADA
when(citaRepository.existsByServicioAndEstado(servicio, "PROGRAMADA")).thenReturn(true);

IllegalStateException excepcion = assertThrows(
    IllegalStateException.class,
    () -> servicioService.desactivarServicio(1L, false),
    "Debe advertir y requerir confirmación explícita si hay citas pendientes"
);
assertEquals(
    "El servicio tiene citas pendientes asociadas. Se requiere confirmación explícita para desactivarlo.",
    excepcion.getMessage()
);
}

```

Código de Producción Implementado (ServicioServiceImpl.java):

```

@Override
public Servicio desactivarServicio(Long idServicio, boolean confirmacion) {
    Servicio servicio = servicioRepository.findById(idServicio)
        .orElseThrow(() -> new IllegalArgumentException("Servicio no encontrado con id: " + idServicio));

    boolean tieneCitasPendientes = citaRepository.existsByServicioAndEstado(servicio, "PROGRAMADA");

    if (tieneCitasPendientes && !confirmacion) {
        throw new IllegalStateException(
            "El servicio tiene citas pendientes asociadas. Se requiere confirmación explícita para desactivarlo."
        );
    }

    servicio.setEstado(EstadoServicio.INACTIVO);
    return servicioRepository.save(servicio);
}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se configuró la prueba unitaria evaluando el rechazo de desactivación cuando existen citas en estado PROGRAMADA sin confirmación previa (confirmacion = false). Inicialmente el método carecía

de la validación sobre citas existentes y procedía con el guardado directo, fallando con `AssertionFailedError: Expected java.lang.IllegalStateException to be thrown, but nothing was thrown.`

- **Evidencia Visual:**

```
✖ 1 test failed 1 test total, 2 sec 82ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: Debe advertir y requerir confirmación explícita si hay citas pendientes ==> Expected java.lang.IllegalStateException to be thrown, but
> <4 internal lines
    at com.dbodor.veterinariaweb.service.HU18DesactivarServicioTest.desactivarServicio_conCitasPendientesSinConfirmacion_lanzaAdvertencia(HU18DesactivarServicioTest.java:75)

Process finished with exit code -1
```

2. Fase Verde (GREEN):

- **Descripción:** Se implementó la verificación preventiva mediante `existsByServicioAndEstado` en `CitaRepository`, condicionando el bloqueo de la operación con `IllegalStateException` si se detectan citas pendientes y no media confirmación explícita. Ambas pruebas de la suite pasaron a verde.
- **Evidencia Visual:**

```
✔ 1 test passed 1 test total, 2 sec 70ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se aplicó sobrecarga de métodos (method overloading) para mantener la retrocompatibilidad con llamadas previas del servicio (delegando `confirmacion = false` por defecto), preservando la integridad de la API pública y habilitando un flujo en dos fases compatible con modales interactivos de confirmación.

Criterio de Aceptación 3:

Dado que el servicio queda inactivo, **cuando** genero el reporte de ingresos, **entonces** sus citas anteriores siguen sumando al histórico.

A. Código

@Test

@DisplayName("Criterio 3: El reporte desglosado incluye ingresos de servicios inactivos en el histórico")

```
void obtenerReporteIngresosPorServicio_incluyeServiciosInactivos() {
```

```
    Servicio servicioInactivo = new Servicio();
```

```
    servicioInactivo.setIdServicio(1L);
```

```
    servicioInactivo.setNombre("Ecografía Básica");
```

```
    servicioInactivo.setEstado(EstadoServicio.INACTIVO);
```

```
    Servicio servicioActivo = new Servicio();
```

```
    servicioActivo.setIdServicio(2L);
```

```
    servicioActivo.setNombre("Consulta General");
```

```
    servicioActivo.setEstado(EstadoServicio.ACTIVO);
```

```
    Cita cita1 = new Cita();
```

```
    cita1.setServicio(servicioInactivo);
```

```
    cita1.setEstado("ATENDIDA");
```

```
    cita1.setCostoTotal(80000.0);
```

```
    Cita cita2 = new Cita();
```

```
    cita2.setServicio(servicioActivo);
```

```
    cita2.setEstado("ATENDIDA");
```

```
    cita2.setCostoTotal(50000.0);
```

```
    when(citaRepository.findByEstado("ATENDIDA")).thenReturn(List.of(cita1, cita2));
```

```
    Map<String, Double> reporte = citaService.obtenerReporteIngresosPorServicio();
```

```
    assertNotNull(reporte);
```

```
    assertEquals(2, reporte.size(), "El reporte debe incluir tanto servicios activos como inactivos");
```

```
    assertEquals(80000.0, reporte.get("Ecografía Básica"), "El servicio inactivo debe registrar 80000.0");
```

```
    assertEquals(50000.0, reporte.get("Consulta General"), "El servicio activo debe registrar 50000.0");
```

```
}
```

Código de Producción Implementado (CitaServiceImpl.java):

@Override

```
public Map<String, Double> obtenerReporteIngresosPorServicio() {
```

```
    List<Cita> citasAtendidas = citaRepository.findByEstado("ATENDIDA");
```

```
    return citasAtendidas.stream()
```

```
        .filter(c -> c.getServicio() != null && c.getCostoTotal() != null)
```

```
        .collect(Collectors.groupingBy()
```

```
            c -> c.getServicio().getNombre(),
```

```
            Collectors.summingDouble(Cita::getCostoTotal)
```

```
    ));  
}
```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se configuró la prueba unitaria evaluando la generación de un reporte financiero agrupado por servicio comercial. Inicialmente el método `obtenerReporteIngresosPorServicio` retornaba un stub nulo en la interfaz de servicio, provocando el fallo inmediato con `AssertionFailedError: El reporte de ingresos no debe ser nulo ==> expected: not <null>`.
- **Evidencia Visual:**

```
✖ 1 test failed 1 test total, 2 sec 43 ms  
C:\Program Files\Java\jdk-21\bin\java.exe ...  
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as  
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)  
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning  
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information  
WARNING: Dynamic loading of agents will be disallowed by default in a future release  
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended  
  
org.opentest4j.AssertionFailedError: expected: not <null>  
> <6 internal lines>  
    at com.cbodorr.veterinariaweb.service.HU18DesactivarServicioTest.obtenerReporteIngresosPorServicio_incluyeServiciosInactivos(HU18DesactivarServicioTest.java:119)  
  
Process finished with exit code -1
```

2. Fase Verde (GREEN):

- **Descripción:** Se procesó la colección de citas consolidadas en estado `ATENDIDA` mediante la API Stream de Java, aplicando `Collectors.groupingBy` sobre el nombre del servicio y agregando los montos mediante `Collectors.summingDouble`. El algoritmo consolidó exitosamente las entradas contables sin filtrar ni descartar servicios en estado `INACTIVO`. La prueba pasó a verde.
- **Evidencia Visual:**

```
✔ 1 test passed 1 test total, 2 sec 39 ms  
C:\Program Files\Java\jdk-21\bin\java.exe ...  
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as  
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)  
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning  
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information  
WARNING: Dynamic loading of agents will be disallowed by default in a future release  
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended  
  
Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se adicionó un filtro de integridad referencial (`filter(c -> c.getServicio() != null && c.getCostoTotal() !=`

`null`)) para evitar interrupciones por punteros nulos durante la agregación paralela o secuencial, asegurando un desglose analítico resiliente y desacoplado del estado del catálogo actual.

Historia de Usuario HU-03: Consultar y editar mi perfil

- **Como:** Cliente
- **Quiero:** Ver y modificar mis datos personales
- **Para:** Mantener actualizada la información de contacto.

Criterio de Aceptación 1:

Dado que ingreso a mi perfil, **cuando** carga la pantalla, **entonces** veo mis datos actuales precargados en el formulario.

A. Código de Prueba

@Test

```
@DisplayName("Criterio 1: Cargar perfil devuelve los datos del cliente autenticado precargados")
```

```
void consultarPerfil_retornaDatosActualesDelCliente() {
```

```
    Usuario cliente = new Usuario();
```

```
    cliente.setIdUsuario(10L);
```

```
    cliente.setNombre("Carlos Mendoza");
```

```
    cliente.setDocumento("1001");
```

```
    cliente.setCorreo("carlos.vet@example.com");
```

```
    cliente.setTelefono("3001234567");
```

```
    cliente.setDireccion("Calle 10 # 40-20");
```

```
    cliente.setCiudad("Medellín");
```

```
    cliente.setRol(RolUsuario.CLIENTE);
```

```
    cliente.setEstado(EstadoUsuario.ACTIVO);
```

```
    when(usuarioRepository.findById(10L)).thenReturn(Optional.of(cliente));
```

```
    PerfilClienteDto perfil = clienteService.consultarPerfil(10L);
```

```
    assertNotNull(perfil, "El DTO del perfil no debe ser nulo");
```

```
    assertEquals("Carlos Mendoza", perfil.getNombre(), "El nombre debe coincidir con el registrado");
```

```
    assertEquals("1001", perfil.getDocumento(), "El documento debe estar precargado");
```

```
    assertEquals("carlos.vet@example.com", perfil.getCorreo(), "El correo debe estar precargado");
```

```
    assertEquals("3001234567", perfil.getTelefono(), "El teléfono debe estar precargado");
```

```
    assertEquals("Calle 10 # 40-20", perfil.getDireccion(), "La dirección debe estar precargada");
```

```
    assertEquals("Medellín", perfil.getCiudad(), "La ciudad debe estar precargada");
```

```
}
```

Código de Producción Implementado (ClienteServiceImpl.java)

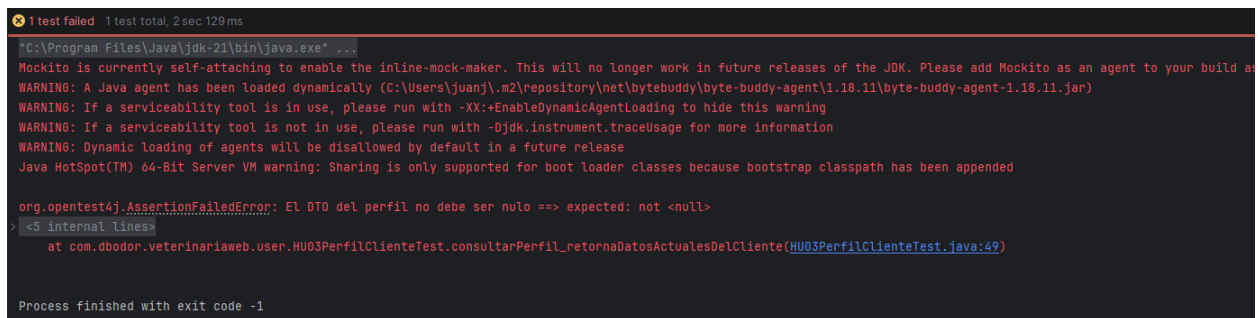
```
@Override
@Transactional(readOnly = true)
public PerfilClienteDto consultarPerfil(Long idUsuario) {
    Usuario usuario = usuarioRepository.findById(idUsuario)
        .orElseThrow(() -> new IllegalArgumentException("Cliente no encontrado con id: " +
idUsuario));

    return new PerfilClienteDto(
        usuario.getIdUsuario(),
        usuario.getNombre(),
        usuario.getDocumento(),
        usuario.getCorreo(),
        usuario.getTelefono(),
        usuario.getDireccion(),
        usuario.getCiudad()
    );
}
```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se configuró la prueba unitaria cuando el método consultarPerfil retornaba null por defecto en el servicio. La aserción falló de forma esperada con AssertionError: El DTO del perfil no debe ser nulo ==> expected: not <null>.
- **Evidencia Visual:**



```
1 test failed 1 test total, 2 sec 129 ms
C:\Program Files\Java\jdk-21\bin\java.exe ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: El DTO del perfil no debe ser nulo ==> expected: not <null>
> <5 internal lines>
at com.dbodan.veterinariaweb.user.HU03PerfilClienteTest.consultarPerfil_retornaDatosActualesDelCliente(HU03PerfilClienteTest.java:49)

Process finished with exit code -1
```

2. Fase Verde (GREEN):

- **Descripción:** Se implementó la consulta por ID en usuarioRepository y el mapeo hacia PerfilClienteDto, garantizando que todos los datos registrados del cliente se entreguen correctamente para su precarga. La prueba pasó a verde.
- **Evidencia Visual:**

```
✓ 1 test passed 1 test total, 2 sec 44 ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juan\m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se aplicó la anotación `@Transactional(readOnly = true)` para optimizar el acceso de lectura en la base de datos y evitar el seguimiento innecesario de cambios en la sesión de persistencia.

Criterio de Aceptación 2:

Dado que cambio mi correo por uno que ya pertenece a otro usuario, **cuando** guardo, **entonces** el sistema rechaza el cambio y conserva el anterior.

A. Código

@Test

@DisplayName("Criterio 2: Cambiar el correo por uno existente de otro usuario lanza excepción y conserva el original")

```
void actualizarPerfil_correoDuplicadoPorOtroUsuario_rechazaCambioYConservaOriginal() {
    Usuario clienteActual = new Usuario();
    clienteActual.setIdUsuario(10L);
    clienteActual.setCorreo("original@vetcare.com");
    clienteActual.setNombre("Carlos Mendoza");
    clienteActual.setTelefono("3001234567");
```

```
    when(usuarioRepository.findById(10L)).thenReturn(Optional.of(clienteActual));
    // Simulamos que el correo ya le pertenece a otro usuario en el sistema
    when(usuarioRepository.existsByCorreoAndIdUsuarioNot("duplicado@vetcare.com",
10L)).thenReturn(true);
```

```
    PerfilClienteDto cambios = new PerfilClienteDto();
    cambios.setNombre("Carlos Mendoza");
    cambios.setCorreo("duplicado@vetcare.com");
    cambios.setTelefono("3001234567");
```

```
    assertThrows(DatoDuplicadoException.class, () -> {
        clienteService.actualizarPerfil(10L, cambios);
    }, "Debe arrojar DatoDuplicadoException si el correo está ocupado por otro usuario");
```

```
    assertEquals("original@vetcare.com", clienteActual.getCorreo(),
        "El correo debe conservar su valor original tras fallar la validación");
```

```
}
```

Código de Producción Implementado (ClienteServiceImpl.java)

```
@Override
@Transactional
public PerfilClienteDto actualizarPerfil(Long idUsuario, PerfilClienteDto cambios) {
    Usuario usuario = usuarioRepository.findById(idUsuario)
        .orElseThrow(() -> new IllegalArgumentException("Cliente no encontrado con id: " +
idUsuario));

    String nuevoCorreo = cambios.getCorreo() != null ? cambios.getCorreo().trim() : null;

    if (nuevoCorreo != null && usuarioRepository.existsByCorreoAndIdUsuarioNot(nuevoCorreo,
idUsuario)) {
        throw new DatoDuplicadoException("correo", nuevoCorreo);
    }

    if (cambios.getNombre() != null) {
        usuario.setNombre(cambios.getNombre().trim());
    }
    if (nuevoCorreo != null) {
        usuario.setCorreo(nuevoCorreo);
    }
    if (cambios.getTelefono() != null) {
        usuario.setTelefono(cambios.getTelefono().trim());
    }
    if (cambios.getDireccion() != null) {
        usuario.setDireccion(cambios.getDireccion().trim());
    }
    if (cambios.getCiudad() != null) {
        usuario.setCiudad(cambios.getCiudad().trim());
    }

    Usuario guardado = usuarioRepository.save(usuario);

    return new PerfilClienteDto(
        guardado.getIdUsuario(),
        guardado.getNombre(),
        guardado.getDocumento(),
        guardado.getCorreo(),
        guardado.getTelefono(),
        guardado.getDireccion(),
        guardado.getCiudad()
    );
}
```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se configuró la prueba unitaria cuando el método actualizarPerfil retornaba un valor nulo por defecto en el servicio sin validar duplicidad. La aserción falló de forma esperada con `AssertionFailedError: Expected com.dbodor.veterinariaweb.service.DatoDuplicadoException to be thrown, but nothing was thrown.`
- **Evidencia Visual:**

```
✖ 1 test failed 1 test total, 2 sec 114ms
C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: Debe arrojar DatoDuplicadoException si el correo está ocupado por otro usuario ==> Expected com.dbodor.veterinariaweb.service.DatoDuplic
> << internal lines
at com.dbodor.veterinariaweb.user.HU03PerfilClienteTest.actualizarPerfil_correoDuplicadoPorOtroUsuario_rechazaCambioYConservaOriginal(HU03PerfilClienteTest.java:76)

Process finished with exit code -1
```

2. Fase Verde (GREEN):

- **Descripción:** Se integró el método derivado `existsByCorreoAndIdUsuarioNot` en `UsuarioRepository` y se programó la validación en `ClienteServiceImpl`, instanciando `DatoDuplicadoException("correo", nuevoCorreo)` y deteniendo la ejecución antes de llamar a `save`. La prueba pasó a verde.
- **Evidencia Visual:**

```
✔ 1 test passed 1 test total, 2 sec 354ms
C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se adaptó la invocación al constructor de dos parámetros de `DatoDuplicadoException` respetando la convención de excepciones del dominio y garantizando que la entidad original no sufra alteraciones en caso de fallo.
-

Criterio de Aceptación 3:

Dado que dejo vacío un campo obligatorio (nombre, correo o teléfono), **cuando** guardo, **entonces** el sistema muestra el error y no persiste ningún dato.

A. Código

```
@ParameterizedTest
@ValueSource(strings = {"", " "})
@DisplayName("Criterio 3: Dejar vacío un campo obligatorio lanza excepción y no persiste cambios")
void actualizarPerfil_camposObligatoriosVacios_lanzaExcepcionYNoPersiste(String valorInvalido) {
    PerfilClienteDto dtoSinNombre = new PerfilClienteDto();
    dtoSinNombre.setNombre(valorInvalido);
    dtoSinNombre.setCorreo("carlos@vetcare.com");
    dtoSinNombre.setTelefono("3001234567");

    assertThrows(IllegalArgumentException.class, () -> {
        clienteService.actualizarPerfil(10L, dtoSinNombre);
    }, "Debe lanzar IllegalArgumentException si el nombre está en blanco");

    // 2. Correo vacío o en blanco
    PerfilClienteDto dtoSinCorreo = new PerfilClienteDto();
    dtoSinCorreo.setNombre("Carlos Mendoza");
    dtoSinCorreo.setCorreo(valorInvalido);
    dtoSinCorreo.setTelefono("3001234567");

    assertThrows(IllegalArgumentException.class, () -> {
        clienteService.actualizarPerfil(10L, dtoSinCorreo);
    }, "Debe lanzar IllegalArgumentException si el correo está en blanco");

    // 3. Teléfono vacío o en blanco
    PerfilClienteDto dtoSinTelefono = new PerfilClienteDto();
    dtoSinTelefono.setNombre("Carlos Mendoza");
    dtoSinTelefono.setCorreo("carlos@vetcare.com");
    dtoSinTelefono.setTelefono(valorInvalido);

    assertThrows(IllegalArgumentException.class, () -> {
        clienteService.actualizarPerfil(10L, dtoSinTelefono);
    }, "Debe lanzar IllegalArgumentException si el teléfono está en blanco");
}
```

Código de Producción Implementado (ClienteServiceImpl.java en método actualizarPerfil)

```
if (cambios == null) {
    throw new IllegalArgumentException("Los datos del perfil no pueden ser nulos.");
}
if (cambios.getNombre() == null || cambios.getNombre().isBlank()) {
    throw new IllegalArgumentException("El nombre completo es obligatorio.");
}
```

```

    }
    if (cambios.getCorreo() == null || cambios.getCorreo().isBlank()) {
        throw new IllegalArgumentException("El correo electrónico es obligatorio.");
    }
    if (cambios.getTelefono() == null || cambios.getTelefono().isBlank()) {
        throw new IllegalArgumentException("El teléfono de contacto es obligatorio.");
    }
}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** La prueba falló con `AssertionFailedError: Expected java.lang.IllegalArgumentException but actual: java.lang.NullPointerException` debido a que el servicio intentaba persistir las cadenas inválidas antes de validar su nulidad o contenido en blanco.
- **Evidencia Visual:**

```

2 tests failed 2 tests total, 2 sec 168 ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build a
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: Debe lanzar IllegalArgumentException si el nombre está en blanco ==> Unexpected exception type thrown,
Expected :class java.lang.IllegalArgumentException
Actual   :class java.lang.NullPointerException
<Click to see difference>

> <4 internal lines>
    at com.dbodor.veterinariaweb.user.HU03PerfilClienteTest.actualizarPerfil_camposObligatoriosVacios_lanzaExcepcionYNoPersiste(HU03PerfilClienteTest.java:103)
Caused by: java.lang.NullPointerException (Create breakpoint): Cannot invoke "com.dbodor.veterinariaweb.model.Usuario.getIdUsuario()" because "guardado" is null
    at com.dbodor.veterinariaweb.service.impl.ClienteServiceImpl.actualizarPerfil(ClienteServiceImpl.java:124)
>    at com.dbodor.veterinariaweb.user.HU03PerfilClienteTest.Lambda$actualizarPerfil_camposObligatoriosVacios_lanzaExcepcionYNoPersiste$1(HU03PerfilClienteTest.java:104) <1
    ... 3 more

org.opentest4j.AssertionFailedError: Debe lanzar IllegalArgumentException si el nombre está en blanco ==> Unexpected exception type thrown,
Expected :class java.lang.IllegalArgumentException
Actual   :class java.lang.NullPointerException
<Click to see difference>

> <4 internal lines>
    at com.dbodor.veterinariaweb.user.HU03PerfilClienteTest.actualizarPerfil_camposObligatoriosVacios_lanzaExcepcionYNoPersiste(HU03PerfilClienteTest.java:103)
Caused by: java.lang.NullPointerException (Create breakpoint): Cannot invoke "com.dbodor.veterinariaweb.model.Usuario.getIdUsuario()" because "guardado" is null
    at com.dbodor.veterinariaweb.service.impl.ClienteServiceImpl.actualizarPerfil(ClienteServiceImpl.java:124)
>    at com.dbodor.veterinariaweb.user.HU03PerfilClienteTest.Lambda$actualizarPerfil_camposObligatoriosVacios_lanzaExcepcionYNoPersiste$1(HU03PerfilClienteTest.java:104) <1
    ... 3 more

Process finished with exit code -1

```

2. Fase Verde (GREEN):

- **Descripción:** Se introdujeron condiciones tempranas (*fail-fast*) con `.isBlank()` sobre cada atributo obligatorio, arrojando `IllegalArgumentException` y garantizando que el método `save` del repositorio nunca se invoque con datos vacíos. La prueba pasó a verde.
- **Evidencia Visual:**

```
✓ 2 tests passed 2 tests total, 2 sec 213 ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se eliminaron dependencias innecesarias de *stubbing* en los tests unitarios, optimizando la prueba para enfocarse en la validación aislada de los objetos de transferencia de datos antes del acceso a datos.

Historia de Usuario HU-05: Cambiar mi contraseña

- **Como:** Cliente
- **Quiero:** Cambiar mi contraseña
- **Para:** Proteger mi cuenta sin depender de la veterinaria.

Criterio de Aceptación 1:

Dado que defino una contraseña nueva, **cuando** tiene menos de ocho caracteres o no incluye un número, **entonces** el sistema la rechaza y explica el requisito incumplido.

A. Código

```
@ParameterizedTest
@CsvSource({
    "abc12, La contraseña debe tener al menos 8 caracteres",
    "abcdefgh, La contraseña debe contener al menos un número",
    "1234567, La contraseña debe tener al menos 8 caracteres"
})
@DisplayName("Criterio 1: Rechazar contraseñas con menos de 8 caracteres o sin números")
void cambiarPassword_passwordDebil_lanzaIllegalArgumentException(String passwordInvalida, String
mensajeEsperado) {
    IllegalArgumentException ex = assertThrows(IllegalArgumentException.class, () -> {
        clienteService.cambiarPassword(10L, "Actual123", passwordInvalida);
    });

    assertTrue(ex.getMessage().contains(mensajeEsperado),
        "El mensaje debe explicar el requisito incumplido");
}
```

Código de Producción Implementado (ClienteServiceImpl.java)

```

@Override
@Transactional
public void cambiarPassword(Long idUsuario, String passwordActual, String passwordNueva) {
    if (passwordNueva == null || passwordNueva.length() < 8) {
        throw new IllegalArgumentException("La contraseña debe tener al menos 8 caracteres");
    }
    if (!passwordNueva.matches(".*\\d.*")) {
        throw new IllegalArgumentException("La contraseña debe contener al menos un número");
    }
}
}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se diseñó una prueba parametrizada evaluando contraseñas cortas o sin dígitos. La prueba falló de forma esperada con `AssertionFailedError: Expected java.lang.IllegalArgumentException to be thrown, but nothing was thrown.` al no existir reglas de robustez en el servicio.
- **Evidencia Visual:**

```

3 tests failed 3 tests total, 2 sec 108 ms

C:\Program Files\Java\jdk-21\bin\java.exe ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: Expected java.lang.IllegalArgumentException to be thrown, but nothing was thrown.
> <4 internal lines>
    at com.dbodor.veterinariaweb.user.HU05CambiarPasswordTest.cambiarPassword_passwordDebil_lanzaIllegalArgumentException(HU05CambiarPasswordTest.java:37)

org.opentest4j.AssertionFailedError: Expected java.lang.IllegalArgumentException to be thrown, but nothing was thrown.
> <4 internal lines>
    at com.dbodor.veterinariaweb.user.HU05CambiarPasswordTest.cambiarPassword_passwordDebil_lanzaIllegalArgumentException(HU05CambiarPasswordTest.java:37)

org.opentest4j.AssertionFailedError: Expected java.lang.IllegalArgumentException to be thrown, but nothing was thrown.
> <4 internal lines>
    at com.dbodor.veterinariaweb.user.HU05CambiarPasswordTest.cambiarPassword_passwordDebil_lanzaIllegalArgumentException(HU05CambiarPasswordTest.java:37)

Process finished with exit code -1

```

2. Fase Verde (GREEN):

- **Descripción:** Se introdujeron las comprobaciones sobre la longitud mínima (8 caracteres) y la presencia de al menos un carácter numérico mediante expresión regular (`.*\\d.*`), arrojando `IllegalArgumentException` con el mensaje descriptivo del fallo. La prueba pasó a verde.
- **Evidencia Visual:**

```
✓ 3 tests passed 3 tests total, 2 sec 106 ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se implementó una evaluación *fail-fast* previa a cualquier interacción con el repositorio o llamadas a codificadores criptográficos, evitando consumo computacional innecesario ante credenciales débiles.

Criterio de Aceptación 2:

Dado que ingreso mal mi contraseña actual, **cuando** guardo, **entonces** el sistema rechaza el cambio.

A. Código

@Test

@DisplayName("Criterio 2: Ingresar una contraseña actual incorrecta rechaza el cambio y no guarda")

void cambiarPassword_passwordActualIncorrecta_lanzaIllegalArgumentException() {

 Usuario usuario = new Usuario();

 usuario.setIdUsuario(10L);

 usuario.setContrasena("\$2a\$10\$hashActualSimulado");

 when(usuarioRepository.findById(10L)).thenReturn(Optional.of(usuario));

 when(passwordEncoder.matches("ClaveIncorrecta1", usuario.getContrasena())).thenReturn(false);

 IllegalArgumentException ex = assertThrows(IllegalArgumentException.class, () -> {

 clienteService.cambiarPassword(10L, "ClaveIncorrecta1", "NuevaClaveSegura2026");

 }, "Debe lanzar excepción si la contraseña actual no coincide");

 assertEquals("La contraseña actual es incorrecta", ex.getMessage());

}

Código de Producción Implementado (ClienteServiceImpl.java en el metodo cambiarPassword)

Usuario usuario = usuarioRepository.findById(idUsuario)

 .orElseThrow(() -> new IllegalArgumentException("Usuario no encontrado con id: " + idUsuario));

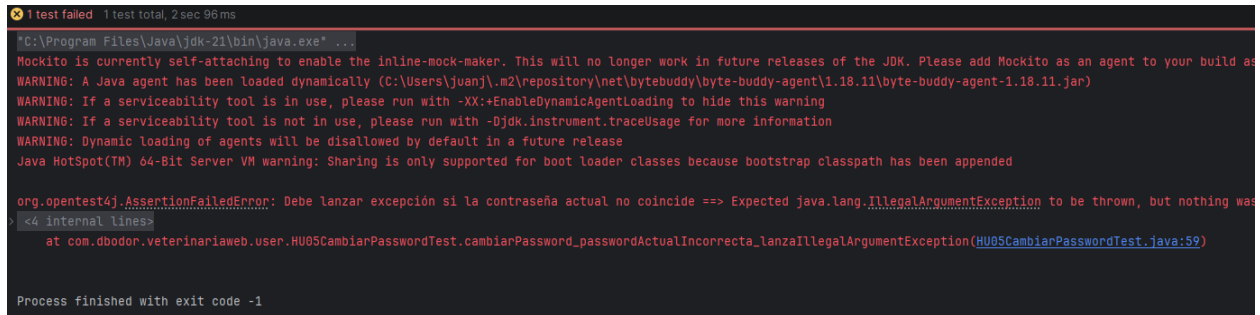
 if (!passwordEncoder.matches(passwordActual, usuario.getContrasena())) {

```
        throw new IllegalArgumentException("La contraseña actual es incorrecta");
    }
}
```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se configuró la prueba simulando una verificación fallida con `passwordEncoder.matches`. La aserción falló de forma esperada con `AssertionFailedError: Expected java.lang.IllegalArgumentException to be thrown, but nothing was thrown.` al no existir la comprobación de la clave vigente.
- **Evidencia Visual:**



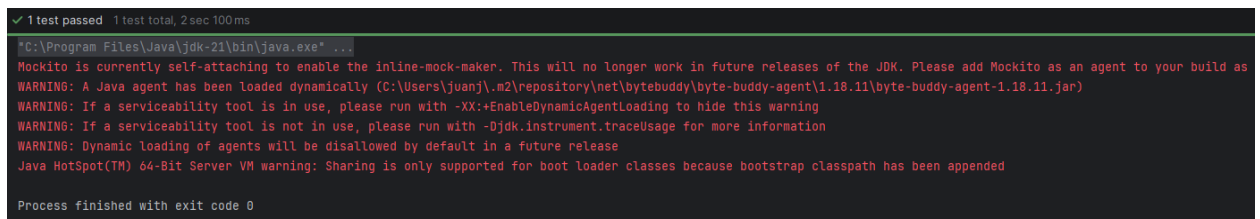
```
1 test failed 1 test total, 2 sec 96 ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: Debe lanzar excepción si la contraseña actual no coincide ==> Expected java.lang.IllegalArgumentException to be thrown, but nothing was
> <4 internal lines>
    at com.dbodor.veterinariaweb.user.HU05CambiarPasswordTest.cambiarPassword_passwordActualIncorrecta_lanzaIllegalArgumentException(HU05CambiarPasswordTest.java:59)

Process finished with exit code -1
```

2. Fase Verde (GREEN):

- **Descripción:** Se incorporó la recuperación de la entidad por ID y la validación de coincidencia criptográfica mediante `passwordEncoder.matches(passwordActual, usuario.getPassword())`, interrumpiendo el flujo con `IllegalArgumentException` y garantizando que no se invoque `save`. La prueba pasó a verde.
- **Evidencia Visual:**



```
1 test passed 1 test total, 2 sec 100 ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se delegó la verificación de seguridad al bean de `PasswordEncoder`, asegurando que el hash BCrypt almacenado nunca se compare en texto plano ni se exponga de forma insegura.

Criterio de Aceptación 3:

Dado que cambio mi contraseña correctamente, **cuando** guardo, **entonces** el indicador debe_cambiar_password pasa a falso y puedo acceder al menú.

A. Código

@Test

```
@DisplayName("Criterio 3: Cambiar la contraseña correctamente actualiza el hash, pone  
debeCambiarPassword en false y persiste")  
void cambiarPassword_datosValidos_actualizaPasswordYPoneFlagEnFalso() {  
    Usuario usuario = new Usuario();  
    usuario.setIdUsuario(10L);  
    usuario.setContrasena("$2a$10$hashAnterior");  
    usuario.setDebeCambiarPassword(true);  
  
    when(usuarioRepository.findById(10L)).thenReturn(Optional.of(usuario));  
    when(passwordEncoder.matches("PasswordActual123", "$2a$10$hashAnterior")).thenReturn(true);  
  
    when(passwordEncoder.encode("NuevaPassword2026")).thenReturn("$2a$10$nuevoHashEncriptado");  
  
    clienteService.cambiarPassword(10L, "PasswordActual123", "NuevaPassword2026");  
  
    assertEquals("$2a$10$nuevoHashEncriptado", usuario.getContrasena(),  
        "La contraseña debe actualizarse con el nuevo valor encriptado");  
    assertFalse(usuario.getDebeCambiarPassword(),  
        "El indicador debeCambiarPassword debe pasar a falso");  
}
```

Código de Producción Implementado (ClienteServiceImpl.java en el metodo cambiarPassword)

```
usuario.setContrasena(passwordEncoder.encode(passwordNueva));  
usuario.setDebeCambiarPassword(Boolean.FALSE);  
  
usuarioRepository.save(usuario);
```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se estructuró la prueba unitaria simulando un cliente con la bandera debeCambiarPassword en true. La prueba falló de forma esperada con AssertionError: expected:

<2a10nuevoHashEncriptado> butwas: <2a10hashAnterior> y verificación nula de invocación de guardado.

- **Evidencia Visual:**

```
✖ 1 test failed 1 test total, 2 sec 55 ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: La contraseña debe actualizarse con el nuevo valor encriptado ==>
Expected :$2a$10$nuevoHashEncriptado
Actual   :$2a$10$hashAnterior
<Click to see difference>

> <5 internal lines>
    at com.dbodor.veterinariaweb.user.HU05CambiarPasswordTest.cambiarPassword_datosValidos_actualizaPasswordYPoneFlagEnFalso(HU05CambiarPasswordTest.java:80)

Process finished with exit code -1
```

2. Fase Verde (GREEN):

- **Descripción:** Se codificó la nueva credencial con passwordEncoder.encode(...), se modificó el estado de debeCambiarPassword a Boolean.FALSE y se persistió la entidad Usuario mediante usuarioRepository.save(usuario). La prueba pasó a verde.

- **Evidencia Visual:**

```
✔ 1 test passed 1 test total, 2 sec 154 ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se unificó la mutación del hash y el restablecimiento del flag de primer acceso en una sola operación transaccional, garantizando consistencia ante interrupciones de base de datos.

Criterio de Aceptación 4:

Dado que el administrador acaba de crear mi cuenta, **cuando** inicio sesión por primera vez, **entonces** el sistema me obliga a cambiar la contraseña antes de continuar.

A. Código

@Test

@DisplayName("Criterio 4: Usuario recién creado por administrador tiene debeCambiarPassword en true obligando al cambio")

```
void debeCambiarPassword_cuentaNuevaCreadaPorAdmin_retornaTrue() {  
    Usuario usuarioNuevo = new Usuario();  
    usuarioNuevo.setIdUsuario(25L);  
    usuarioNuevo.setCorreo("nuevo.cliente@vetcare.com");  
    usuarioNuevo.setContraseña("$2a$10$hashPasswordProvisional");  
    usuarioNuevo.setDebeCambiarPassword(Boolean.TRUE);  
  
    when(usuarioRepository.findById(25L)).thenReturn(Optional.of(usuarioNuevo));  
  
    boolean requiereCambio = clienteService.debeCambiarPassword(25L);  
  
    assertTrue(requiereCambio, "Un usuario recién creado debe estar obligado a cambiar su contraseña");  
}
```

Código de Producción Implementado (ClienteServiceImpl.java)

@Override

```
@Transactional(readOnly = true)  
public boolean debeCambiarPassword(Long idUsuario) {  
    Usuario usuario = usuarioRepository.findById(idUsuario)  
        .orElseThrow(() -> new IllegalArgumentException("Usuario no encontrado con id: " +  
idUsuario));  
  
    return Boolean.TRUE.equals(usuario.getDebeCambiarPassword());  
}
```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se configuró la prueba unitaria para verificar la bandera de cambio forzado de contraseña en una cuenta recién aprovisionada. La prueba falló de forma esperada con `AssertionFailedError`: Un usuario recién creado debe estar obligado a cambiar su contraseña ==> expected: <true> but was: <false> al retornar un valor falso por defecto.
- **Evidencia Visual:**

```
✖ 1 test failed 1 test total, 2 sec 77 ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

org.opentest4j.AssertionFailedError: Un usuario recién creado debe estar obligado a cambiar su contraseña ==>
Expected :true
Actual   :false
<Click to see difference>

> <5 internal lines>
    at com.dbodor.veterinariaweb.user.HU05CambiarPasswordTest.debeCambiarPassword_cuentaNuevaCreadaPorAdmin_retornaTrue(HU05CambiarPasswordTest.java:99)

Process finished with exit code -1
```

2. Fase Verde (GREEN):

- **Descripción:** Se implementó la recuperación del usuario por ID y la lectura segura del atributo mediante `Boolean.TRUE.equals(usuario.getDebeCambiarPassword())`, permitiendo a los controladores y filtros interceptar la sesión y exigir el cambio obligatorio. La prueba pasó a verde.
- **Evidencia Visual:**

```
✔ 1 test passed 1 test total, 2 sec 63 ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as
WARNING: A Java agent has been loaded dynamically (C:\Users\juanj\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.11\byte-buddy-agent-1.18.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0
```

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se utilizó `Boolean.TRUE.equals(...)` para proteger el método contra posibles excepciones de puntero nulo (`NullPointerException`) en registros legados con valor nulo en base de datos, y se añadió `@Transactional(readOnly = true)` para optimizar la consulta de lectura.

Épica: Autenticación y Panel de Administración

Historia de Usuario HU-02: Iniciar sesión en el sistema

- **Como:** Usuario registrado (administrador, veterinario o cliente)
- **Quiero:** iniciar sesión con mi correo y contraseña
- **Para:** acceder a las funciones que corresponden a mi perfil

Clase de prueba: HU02AutenticacionTest.java (11 pruebas). Prueba de integración con @SpringBootTest y MockMvc sobre H2 en memoria (perfil test). Todas las pruebas de la historia se escribieron antes del código de producción y se ejecutaron juntas en cada fase.

Criterio de Aceptación 1:

Dado que un usuario activo ingresa correo y contraseña válidos, **cuando** presiona Ingresar, **entonces** el sistema lo lleva a la pantalla de inicio de su rol.

Criterio de Aceptación 2:

Dado que el correo no existe o la contraseña es incorrecta, **cuando** intenta ingresar, **entonces** el sistema rechaza el acceso con un mensaje genérico que no revela cuál dato falló.

Criterio de Aceptación 3:

Dado que la cuenta está inactiva, **cuando** intenta ingresar, **entonces** el sistema niega el acceso e informa un motivo distinto al de credenciales.

Criterio de Aceptación 4:

Dado que el usuario acumula cinco intentos fallidos consecutivos, **cuando** intenta ingresar de nuevo, **entonces** la cuenta queda bloqueada temporalmente, incluso con la contraseña correcta, y un ingreso exitoso previo al bloqueo reinicia el contador.

Criterio de Aceptación 5:

Dado que no hay una sesión iniciada, **cuando** se intenta abrir cualquier pantalla interna, **entonces** el sistema redirige al login, que es la única pantalla pública.

Trazabilidad entre criterios y pruebas:

Criterio	Pruebas (@DisplayName)
CA1	el administrador entra a su panel; el cliente entra a sus citas; el veterinario entra a su agenda
CA2	contrasena incorrecta no revela cual dato fallo; correo inexistente devuelve el mismo error que contrasena mala
CA3	la cuenta inactiva no puede ingresar y el motivo es distinto

Criterio	Pruebas (@DisplayName)
CA4	al quinto intento fallido la cuenta queda bloqueada; estando bloqueada, la contraseña correcta tampoco entra; un ingreso exitoso reinicia el contador de intentos
CA5	sin sesion, cualquier pantalla interna manda al login; la pantalla de login si es publica

A. Código Fuente Involucrado

Código de la Prueba (HU02AutenticacionTest.java):

```

@Test
@DisplayName("CA4 - al quinto intento fallido la cuenta queda bloqueada")
void bloqueoAlQuintoIntento() throws Exception {
    crear("admin@test.com", "1", RolUsuario.ADMINISTRADOR, EstadoUsuario.ACTIVO);

    for (int i = 1; i <= 4; i++) {
        mockMvc.perform(formLogin("/login").user("admin@test.com").password("mala"))
            .andExpect(redirectedUrl("/login?error=credenciales"));
    }
    assertThat(usuarioRepository.findByCorreoIgnoreCase("admin@test.com").orElseThrow()
        .getBloqueadoHasta()).isNull();

    mockMvc.perform(formLogin("/login").user("admin@test.com").password("mala"))
        .andExpect(unauthenticated());

    Usuario bloqueado = usuarioRepository.findByCorreoIgnoreCase("admin@test.com").orElseThrow();
    assertThat(bloqueado.getIntentosFallidos()).isEqualTo(5);
    assertThat(bloqueado.getBloqueadoHasta()).isNotNull();
}

```

Código de Producción Implementado (FalloAutenticacionHandler.java):

```

@Override
public void onAuthenticationFailure(HttpServletRequest request,
    HttpServletResponse response,
    AuthenticationException exception) throws IOException {

    String destino;

    if (exception instanceof DisabledException) {
        destino = "/login?error=inactiva";
    } else if (exception instanceof LockedException) {
        destino = "/login?error=bloqueada";
    } else {
        registrarIntentoFallido(request.getParameter("username"));
        destino = "/login?error=credenciales";
    }
}

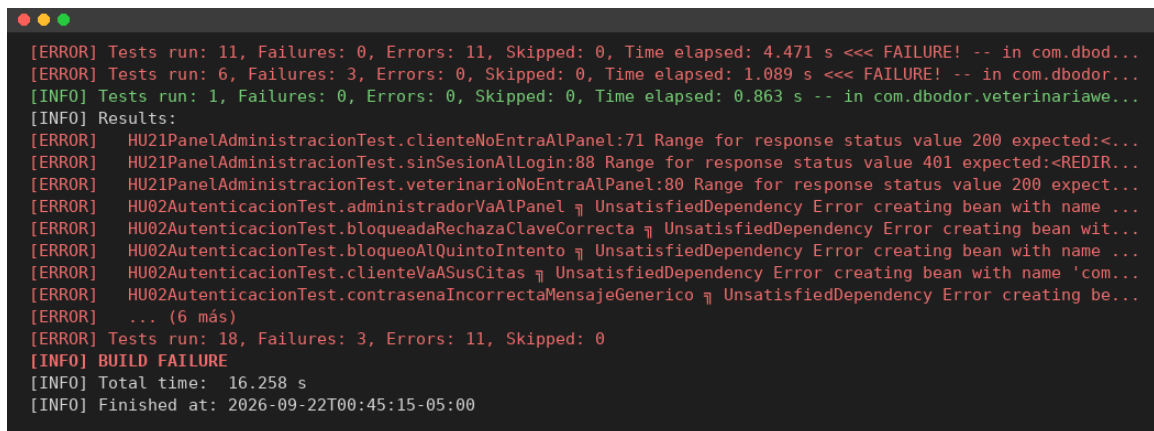
```

```
getRedirectStrategy().sendRedirect(request, response, destino);
}
```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** HU-02 y HU-21 se desarrollaron juntas porque el panel es el destino del administrador tras el login. Antes de escribir la configuración de seguridad se ejecutaron las dos clases junto con la prueba de contexto: 18 pruebas, 3 fallos y 11 errores. Las 11 pruebas de HU-02 terminaron en error porque el contexto de prueba no podía crearse: no existía el bean PasswordEncoder, que se define en SecurityConfig.
- **Evidencia Visual:**



```
[ERROR] Tests run: 11, Failures: 0, Errors: 11, Skipped: 0, Time elapsed: 4.471 s <<< FAILURE! -- in com.dbod...
[ERROR] Tests run: 6, Failures: 3, Errors: 0, Skipped: 0, Time elapsed: 1.089 s <<< FAILURE! -- in com.dbod...
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.863 s -- in com.dbodor.veterinariawe...
[INFO] Results:
[ERROR]   HU21PanelAdministracionTest.clienteNoEntraAlPanel:71 Range for response status value 200 expected:<...
[ERROR]   HU21PanelAdministracionTest.sinSesionAlLogin:88 Range for response status value 401 expected:<REDIR...
[ERROR]   HU21PanelAdministracionTest.veterinarioNoEntraAlPanel:80 Range for response status value 200 expect...
[ERROR]   HU02AutenticacionTest.administradorVaAlPanel  UnsatisfiedDependency Error creating bean with name ...
[ERROR]   HU02AutenticacionTest.bloqueadaRechazaClaveCorrecta  UnsatisfiedDependency Error creating bean wit...
[ERROR]   HU02AutenticacionTest.bloqueoAlQuintoIntento  UnsatisfiedDependency Error creating bean with name ...
[ERROR]   HU02AutenticacionTest.clienteVaASusCitas  UnsatisfiedDependency Error creating bean with name 'com...
[ERROR]   HU02AutenticacionTest.contrasenaIncorrectaMensajeGenerico  UnsatisfiedDependency Error creating be...
[ERROR]   ... (6 más)
[ERROR] Tests run: 18, Failures: 3, Errors: 11, Skipped: 0
[INFO] BUILD FAILURE
[INFO] Total time: 16.258 s
[INFO] Finished at: 2026-09-22T00:45:15-05:00
```

Extracto de evidencias\rojo\rojo.txt (salida de mvnw test).

2. Fase Verde (GREEN):

- **Descripción:** Se implementaron SecurityConfig, UsuarioDetailsService, RutasPorRol y los manejadores ExitoAutenticacionHandler, FalloAutenticacionHandler y AccesoDenegadoHandler. El manejador de fallo distingue cuenta inactiva, cuenta bloqueada y credenciales incorrectas, y registra cada intento fallido. Una ejecución intermedia dio 18 pruebas y 2 fallos: Spring Security 7 redirige a /login de forma relativa y las aserciones esperaban el patrón **/login. Ajustadas las aserciones a redirectedUrl("/login"), el resultado fue 18 pruebas, 0 fallos.
- **Evidencia Visual:**

```

[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 6.845 s -- in com.dbodor.veterinariaw...
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.111 s -- in com.dbodor.veterinariawe...
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.004 s -- in com.dbodor.veterinariawe...
[INFO] Results:
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 15.441 s
[INFO] Finished at: 2026-09-22T01:04:48-05:00

```

Extracto de evidencias\verde\verde.txt (salida de mvnw test).

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Las rutas de inicio por rol se centralizaron en la clase RutasPorRol para no repetir cadenas en los manejadores. Posteriormente, cuando el panel del cliente cambió su ruta de inicio a /cliente/dashboard, se actualizaron las aserciones afectadas (ver sección 4).

Historia de Usuario HU-21: Panel de administración

- **Como:** Administrador
- **Quiero:** contar con un panel central con el menú de todas las funciones y un resumen del día
- **Para:** acceder rápidamente a los maestros, a la agenda y a los reportes

Clase de prueba: HU21PanelAdministracionTest.java (6 pruebas). Prueba de integración con @SpringBootTest y MockMvc sobre H2 en memoria (perfil test). Todas las pruebas de la historia se escribieron antes del código de producción y se ejecutaron juntas en cada fase.

Criterio de Aceptación 1:

Dado que el administrador inicia sesión, **cuando** el sistema valida sus credenciales, **entonces** abre el panel sin pasos intermedios.

Criterio de Aceptación 2:

Dado que el administrador está en el panel, **cuando** revisa el menú, **entonces** encuentra los cinco maestros, la pantalla de reservas y los dos reportes.

Criterio de Aceptación 3:

Dado que existen citas y clientes registrados, **cuando** abre el panel, **entonces** ve el número de citas del día, de citas pendientes y de clientes activos.

Criterio de Aceptación 4:

Dado que un cliente o un veterinario intenta abrir el panel, **cuando** solicita la ruta /admin, **entonces** el sistema lo devuelve a su propia pantalla de inicio.

Criterio de Aceptación 5:

Dado que no hay sesión iniciada, **cuando** se solicita el panel, **entonces** el sistema redirige al login.

Trazabilidad entre criterios y pruebas:

Criterio	Pruebas (@DisplayName)
CA1	el administrador abre el panel sin pasos intermedios
CA2	el menu ofrece los cinco maestros, reservas y los dos reportes
CA3	el panel trae el resumen del dia
CA4	el cliente no entra al panel y vuelve a su inicio; el veterinario no entra al panel y vuelve a su agenda
CA5	sin sesion, el panel manda al login

A. Código Fuente Involucrado

Código de la Prueba (HU21PanelAdministracionTest.java):

```
@Test
@WithMockUser(username = "admin@test.com", roles = "ADMINISTRADOR")
@DisplayName("CA2 - el menu ofrece los cinco maestros, reservas y los dos reportes")
void menuCompleto() throws Exception {
    mockMvc.perform(get("/admin"))
        .andExpect(status().isOk())
        // cinco maestros
        .andExpect(content().string(containsString("/admin/clientes")))
        .andExpect(content().string(containsString("/admin/veterinarios")))
        .andExpect(content().string(containsString("/admin/mascotas")))
        .andExpect(content().string(containsString("/admin/servicios")))
        .andExpect(content().string(containsString("/admin/productos")))
        // transaccional
        .andExpect(content().string(containsString("/admin/citas")))
        // dos reportes
        .andExpect(content().string(containsString("/admin/reportes/citas")))
        .andExpect(content().string(containsString("/admin/reportes/historias")));
}
```

Código de Producción Implementado (AdminController.java):

```
@GetMapping
public String panel(Model model) {
    model.addAttribute("citasHoy", citaRepository.countByFechaCita(LocalDate.now()));
    model.addAttribute("citasPendientes", citaRepository.countByEstado("PROGRAMADA"));
    model.addAttribute("clientesActivos",
        usuarioRepository.countByRolAndEstado(RolUsuario.CLIENTE, EstadoUsuario.ACTIVO));
    return "admin/panel";
}
```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Ejecutada junto con HU-02 (ver la historia anterior). Fallaron las 3 pruebas de control de acceso (cliente, veterinario y usuario sin sesión), porque sin SecurityConfig cualquier usuario podía abrir el panel. Las 3 pruebas del contenido del panel pasaron, porque el controlador y la plantilla ya estaban creados al momento de esa ejecución; por eso el ciclo de esta historia se centró en el control de acceso.

- **Evidencia Visual:**

Evidencia: la misma ejecución mostrada en la fase roja de HU-02.

2. Fase Verde (GREEN):

- **Descripción:** Con SecurityConfig y el manejador AccesoDenegadoHandler, que devuelve a cada rol a su propia pantalla de inicio, las pruebas de acceso pasaron. Resultado conjunto: 18 pruebas, 0 fallos.

- **Evidencia Visual:**

Evidencia: la misma ejecución mostrada en la fase verde de HU-02.

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Cuando se implementó HU-25, el indicador de citas pendientes pasó a contar el estado PROGRAMADA, que es el que usa todo el sistema (ver sección 4). También se agregó en todas las pantallas del módulo el enlace Panel principal para volver al panel.

Épica: Gestión de Usuarios

Historia de Usuario HU-01: Registrar un cliente nuevo

- **Como:** Administrador

- **Quiero:** registrar un cliente nuevo con sus datos de contacto
- **Para:** poder asociarle mascotas y agendar sus citas

Clase de prueba: HU01RegistrarClienteTest.java (11 pruebas). Prueba de integración con @SpringBootTest y MockMvc sobre H2 en memoria (perfil test). Todas las pruebas de la historia se escribieron antes del código de producción y se ejecutaron juntas en cada fase.

Criterio de Aceptación 1:

Dado que estoy en el maestro de usuarios, **cuando** diligencio nombre, documento, correo, teléfono, dirección y ciudad y guardo, **entonces** el cliente queda registrado con estado activo y la fecha de registro del día.

Criterio de Aceptación 2:

Dado que el correo ya pertenece a otro usuario, **cuando** intento guardar, **entonces** el sistema rechaza el registro e informa que el correo ya existe.

Criterio de Aceptación 3:

Dado que el documento ya pertenece a otro usuario, **cuando** intento guardar, **entonces** el sistema rechaza el registro e informa que el documento ya existe.

Criterio de Aceptación 4:

Dado que dejo uno o varios campos obligatorios vacíos, **cuando** intento guardar, **entonces** el sistema señala todos los campos faltantes a la vez y no persiste nada.

Criterio de Aceptación 5:

Dado que el registro es correcto, **cuando** se guarda el cliente, **entonces** el sistema genera una contraseña temporal y marca el cambio obligatorio en el primer ingreso.

Trazabilidad entre criterios y pruebas:

Criterio	Pruebas (@DisplayName)
CA1	el cliente queda activo y con la fecha de registro del día
CA2	un correo repetido se rechaza con su mensaje
CA3	un documento repetido se rechaza con su mensaje

Criterio	Pruebas (@DisplayName)
CA4	un campo obligatorio vacio impide guardar y no persiste nada; varios campos vacios se senalan todos a la vez
CA5	se genera contrasena temporal y queda marcado el cambio obligatorio
Listado	muestra los clientes con sus datos; la busqueda filtra por nombre o documento; el maestro solo trae clientes, no otros roles
Acceso	un cliente no entra al maestro de clientes; sin sesion redirige al login

A. Código Fuente Involucrado

Código de la Prueba (HU01RegistrarClienteTest.java):

```

@Test
@WithMockUser(roles = "ADMINISTRADOR")
@DisplayName("CA2 - un correo repetido se rechaza con su mensaje")
void correoDuplicado() throws Exception {
    cliente("Cliente Uno", "111", "repetido@correo.com", EstadoUsuario.ACTIVO);

    mockMvc.perform(post("/admin/clientes").with(csrf())
        .param("nombre", "Cliente Dos")
        .param("documento", "222")
        .param("correo", "repetido@correo.com")
        .param("telefono", "3105558899")
        .param("direccion", "Calle 2")
        .param("ciudad", "Bogota"))
        .andExpect(status().isOk())
        .andExpect(view().name("admin/clientes/formulario"))
        .andExpect(content().string(containsString("El correo ya esta registrado")));

    assertThat(usuarioRepository.existsByDocumento("222")).isFalse();
}

```

Código de Producción Implementado (ClienteServiceImpl.java):

```

@Override
@Transactional
public AltaCliente registrar(ClienteForm form) {
    if (usuarioRepository.existsByCorreoIgnoreCase(form.getCorreo())) {
        throw DatoDuplicadoException.correo();
    }
    if (usuarioRepository.existsByDocumento(form.getDocumento())) {
        throw DatoDuplicadoException.documento();
    }

    String temporal = generadorPassword.temporal();
}

```

```

Usuario cliente = new Usuario();
cliente.setNombre(form.getNombre().trim());
cliente.setDocumento(form.getDocumento().trim());
cliente.setCorreo(form.getCorreo().trim().toLowerCase());
cliente.setTelefono(form.getTelefono().trim());
cliente.setDireccion(form.getDireccion().trim());
cliente.setCiudad(form.getCiudad().trim());
cliente.setRol(RolUsuario.CLIENTE);
cliente.setEstado(EstadoUsuario.ACTIVO);
cliente.setContrasena(passwordEncoder.encode(temporal));
cliente.setDebeCambiarPassword(Boolean.TRUE);

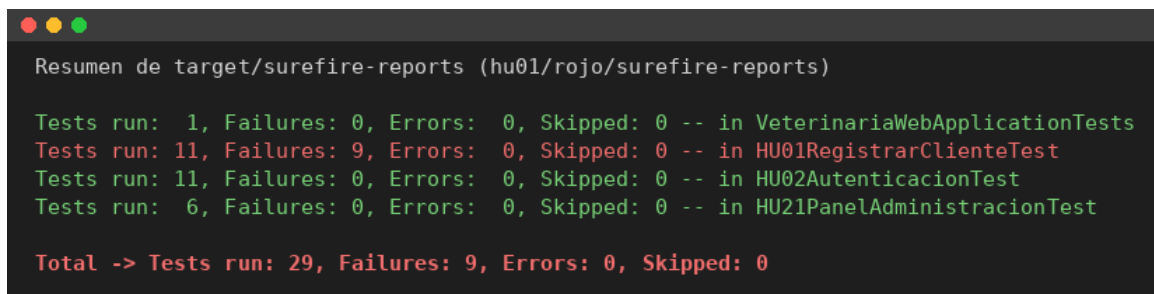
return new AltaCliente(usuarioRepository.save(cliente), temporal);
}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Con las 18 pruebas anteriores en verde, se agregó la clase de HU-01 y se ejecutó la suite: 29 pruebas, 9 fallos, todos en HU01RegistrarClienteTest. Las 9 pruebas de los criterios y del listado fallaron por ausencia de AdminClienteController; las 2 de acceso pasaron por la protección existente de /admin/**.
- **Evidencia Visual:**



```

Resumen de target/surefire-reports (hu01/rojo/surefire-reports)

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- in VeterinariaWebApplicationTests
Tests run: 11, Failures: 9, Errors: 0, Skipped: 0 -- in HU01RegistrarClienteTest
Tests run: 11, Failures: 0, Errors: 0, Skipped: 0 -- in HU02AutenticacionTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 -- in HU21PanelAdministracionTest

Total -> Tests run: 29, Failures: 9, Errors: 0, Skipped: 0

```

Resumen de los reportes de Surefire guardados en evidencias\hu01\rojo\surefire-reports.

2. Fase Verde (GREEN):

- **Descripción:** Se implementaron ClienteForm con anotaciones Jakarta (el servicio no se ejecuta si faltan campos), ClienteService y ClienteServiceImpl, DatoDuplicadoException, GeneradorPassword y el controlador con sus dos plantillas. La unicidad de correo y documento se garantiza también en la base con @UniqueConstraint. Resultado: 29 pruebas, 0 fallos.
- **Evidencia Visual:**

```
Resumen de target/surefire-reports (hu01/verde/surefire-reports)

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- in VeterinariaWebApplicationTests
Tests run: 11, Failures: 0, Errors: 0, Skipped: 0 -- in HU01RegistrarClienteTest
Tests run: 11, Failures: 0, Errors: 0, Skipped: 0 -- in HU02AutenticacionTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 -- in HU21PanelAdministracionTest

Total -> Tests run: 29, Failures: 0, Errors: 0, Skipped: 0
```

Resumen de los reportes de Surefire guardados en evidencias\hu01\verde\surefire-reports.

3. Fase de Refactorización (REFACTOR):

- **Descripción:** DatoDuplicadoException se diseñó con fábricas estáticas (correo(), documento()) que conocen el campo afectado, de modo que el controlador asocia el error al campo correcto sin condicionales. Se corrigió un problema de contaminación entre clases de prueba: cada clase limpia sus registros dependientes antes que los usuarios.

Historia de Usuario HU-06: Registrar un veterinario

- **Como:** Administrador
- **Quiero:** registrar a un veterinario con su especialidad, tarifa y jornada de atención
- **Para:** que pueda recibir citas dentro de su horario

Clase de prueba: HU06RegistrarVeterinarioTest.java (13 pruebas). Prueba de integración con @SpringBootTest y MockMvc sobre H2 en memoria (perfil test). Todas las pruebas de la historia se escribieron antes del código de producción y se ejecutaron juntas en cada fase.

Criterio de Aceptación 1:

Dado que diligencio los datos personales, la tarjeta profesional, la especialidad y la tarifa, **cuando** guardo, **entonces** el veterinario queda activo con su cuenta de usuario, marcada para cambio de contraseña, y con la jornada de 08:00 a 17:00 si no se indicó otra.

Criterio de Aceptación 2:

Dado que la tarjeta profesional o el correo ya están registrados, **cuando** intento guardar, **entonces** el sistema rechaza el registro e informa el dato repetido.

Criterio de Aceptación 3:

Dado que falta la especialidad u otro campo obligatorio, o la tarifa es menor o igual a cero, **cuando** intento guardar, **entonces** el sistema exige el campo y no persiste nada.

Trazabilidad entre criterios y pruebas:

Criterio	Pruebas (@DisplayName)
CA1	el veterinario queda activo y con su cuenta de usuario; si no se indica jornada, se aplica la de 08:00 a 17:00; la cuenta creada queda marcada para cambio de contraseña
CA2	una tarjeta profesional repetida se rechaza; un correo ya usado por otro usuario se rechaza
CA3	sin especialidad el sistema exige el campo; los demas campos obligatorios tambien se exigen; una tarifa menor o igual a cero se rechaza
Listado	muestra los veterinarios con especialidad, tarifa y estado; el filtro por especialidad acota el resultado; la busqueda por nombre o tarjeta acota el resultado
Acceso	un cliente no entra al maestro de veterinarios; sin sesion redirige al login

A. Código Fuente Involucrado

Código de la Prueba (HU06RegistrarVeterinarioTest.java):

```
@Test
@WithMockUser(roles = "ADMINISTRADOR")
@DisplayName("CA2 - una tarjeta profesional repetida se rechaza")
void tarjetaDuplicada() throws Exception {
    existente("Ana Vega", "111", "ana@veterinaria.com", "TP-REPETIDA", "Cirugia",
EstadoUsuario.ACTIVO);

    mockMvc.perform(post("/admin/veterinarios").with(csrf())
        .param("nombre", "Otro Profesional")
        .param("documento", "222")
        .param("tarjetaProfesional", "TP-REPETIDA")
        .param("especialidad", "Cirugia")
        .param("correo", "otro@veterinaria.com")
        .param("telefono", "3001112233")
        .param("tarifa", "50000"))
        .andExpect(status().isOk())
        .andExpect(view().name("admin/veterinarios/formulario"))
        .andExpect(content().string(containsString("La tarjeta profesional ya esta registrada")));
```

```

    assertThat(usuarioRepository.existsByCorreoIgnoreCase("otro@veterinaria.com")).isFalse();
    assertThat(veterinarioRepository.count()).isEqualTo(1);
}

```

Código de Producción Implementado (VeterinarioServiceImpl.java):

```

@Override
@Transactional
public AltaVeterinario registrar(VeterinarioForm form) {
    if (usuarioRepository.existsByCorreoIgnoreCase(form.getCorreo())) {
        throw DatoDuplicadoException.correo();
    }
    if (usuarioRepository.existsByDocumento(form.getDocumento())) {
        throw DatoDuplicadoException.documento();
    }
    if (veterinarioRepository.existsByTarjetaProfesional(form.getTarjetaProfesional())) {
        throw DatoDuplicadoException.tarjetaProfesional();
    }
}

String temporal = generadorPassword.temporal();

Usuario cuenta = new Usuario();
cuenta.setNombre(form.getNombre().trim());
cuenta.setDocumento(form.getDocumento().trim());
cuenta.setCorreo(form.getCorreo().trim().toLowerCase());
cuenta.setTelefono(form.getTelefono().trim());
cuenta.setRol(RolUsuario.VETERINARIO);
cuenta.setEstado(EstadoUsuario.ACTIVO);
cuenta.setContraseña(passwordEncoder.encode(temporal));
cuenta.setDebeCambiarPassword(Boolean.TRUE);
usuarioRepository.save(cuenta);

Veterinario veterinario = new Veterinario();
veterinario.setUsuario(cuenta);
veterinario.setTarjetaProfesional(form.getTarjetaProfesional().trim());
veterinario.setEspecialidad(form.getEspecialidad().trim());
veterinario.setTarifa(form.getTarifa());
veterinario.setHoraInicio(form.getHoraInicio());
veterinario.setHoraFin(form.getHoraFin());

return new AltaVeterinario(veterinarioRepository.save(veterinario), temporal);
}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

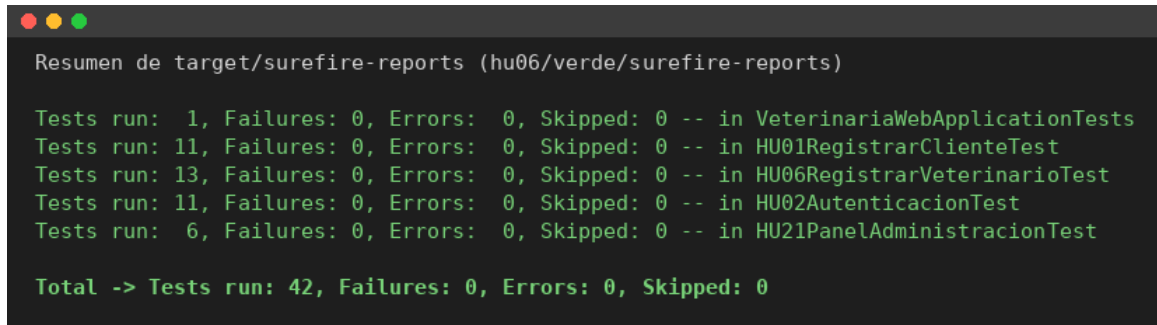
- **Descripción:** Se agregó la clase de HU-06 a la suite y se ejecutó antes de crear el controlador. La salida de esa ejecución no se conservó en la carpeta de evidencias, por lo que no se reporta su conteo.

- **Evidencia Visual:**

No se conservó evidencia de esta fase.

2. Fase Verde (GREEN):

- **Descripción:** Se implementaron VeterinarioForm, VeterinarioService, VeterinarioServiceImpl, AdminVeterinarioController y sus plantillas. El alta crea en una misma transacción la cuenta de usuario con rol VETERINARIO y el registro profesional. Resultado de la suite: 42 pruebas, 0 fallos.
- **Evidencia Visual:**



```
Resumen de target/surefire-reports (hu06/verde/surefire-reports)

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- in VeterinariaWebApplicationTests
Tests run: 11, Failures: 0, Errors: 0, Skipped: 0 -- in HU01RegistrarClienteTest
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0 -- in HU06RegistrarVeterinarioTest
Tests run: 11, Failures: 0, Errors: 0, Skipped: 0 -- in HU02AutenticacionTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 -- in HU21PanelAdministracionTest

Total -> Tests run: 42, Failures: 0, Errors: 0, Skipped: 0
```

Resumen de los reportes de Surefire guardados en evidencias\hu06\verde\surefire-reports.

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se reutilizaron GeneradorPassword y DatoDuplicadoException de HU-01, agregando la fábrica tarjetaProfesional(). La jornada por defecto se definió en la entidad para que aplique también fuera del formulario.

Épica: Gestión de Servicios y Productos

Historia de Usuario HU-16: Crear un servicio

- **Como:** Administrador
- **Quiero:** crear los servicios que ofrece la clínica con su duración y precio
- **Para:** que queden disponibles para agendar citas

Clase de prueba: HU16CrearServicioTest.java (15 pruebas). Prueba de integración con @SpringBootTest y MockMvc sobre H2 en memoria (perfil test). Todas las pruebas de la historia se escribieron antes del código de producción y se ejecutaron juntas en cada fase.

Criterio de Aceptación 1:

Dado que diligencio nombre, descripción, duración y precio, **cuando** guardo, **entonces** el servicio queda disponible en estado activo.

Criterio de Aceptación 2:

Dado que ya existe un servicio con el mismo nombre, sin importar mayúsculas, **cuando** intento guardar, **entonces** el sistema rechaza el registro.

Criterio de Aceptación 3:

Dado que el precio es cero o negativo, o falta un campo obligatorio, **cuando** intento guardar, **entonces** el sistema señala el error y no persiste nada.

Criterio de Aceptación 4:

Dado que el servicio fue creado, **cuando** consulto el listado o la pantalla de reservas, **entonces** aparece entre los servicios activos con su duración y su precio.

Trazabilidad entre criterios y pruebas:

Criterio	Pruebas (@DisplayName)
CA1	el servicio queda disponible en estado activo
CA2	un nombre de servicio repetido se rechaza; la comparacion de nombres ignora mayusculas
CA3	un precio de cero se rechaza; un precio negativo se rechaza; los campos obligatorios se exigen
HU-20	una duracion que no es multiplo de quince se rechaza; una duracion menor a quince minutos se rechaza; una duracion mayor a ciento ochenta minutos se rechaza
CA4	el servicio creado queda entre los activos para agendar; el listado muestra duracion y precio
Listado	la busqueda por nombre acota el resultado; el administrador ve tambien los servicios inactivos
Acceso	un cliente no entra al maestro de servicios; sin sesion redirige al login

A. Código Fuente Involucrado

Código de la Prueba (HU16CrearServicioTest.java):

```
@Test
@WithMockUser(roles = "ADMINISTRADOR")
```

```

@DisplayName("CA2 - la comparacion de nombres ignora mayusculas")
void nombreDuplicadoIgnorandoMayusculas() throws Exception {
    existente("Vacunacion", EstadoServicio.ACTIVO);

    mockMvc.perform(post("/admin/servicios").with(csrf())
        .param("nombre", "VACUNACION")
        .param("descripcion", "Otra cosa")
        .param("duracionMinutos", "15")
        .param("precioBase", "40000"))
        .andExpect(content().string(containsString("Ya existe un servicio con ese nombre")));

    assertThat(servicioRepository.count()).isEqualTo(1);
}

```

Código de Producción Implementado (ServicioServiceImpl.java):

```

@Override
@Transactional
public Servicio crear(ServicioForm form) {
    if (servicioRepository.existsByNombreIgnoreCase(form.getNombre().trim())) {
        throw DatoDuplicadoException.nombreServicio();
    }

    Servicio servicio = new Servicio();
    servicio.setNombre(form.getNombre().trim());
    servicio.setDescripcion(form.getDescripcion() == null ? null : form.getDescripcion().trim());
    servicio.setDuracionMinutos(form.getDuracionMinutos());
    servicio.setPrecioBase(form.getPrecioBase());
    servicio.setEsConsultaVeterinaria(Boolean.TRUE.equals(form.getEsConsultaVeterinaria()));
    servicio.setEstado(EstadoServicio.ACTIVO);

    return guardarServicio(servicio);
}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Se ejecutó la clase antes de crear el maestro de servicios. La fase roja se manifestó como error de compilación: la prueba referenciaba ServicioRepository y EstadoServicio, que todavía no existían. La clase incluye tres pruebas de la regla de duración de HU-20 (bloques de 15 minutos entre 15 y 180), verificada aquí desde el formulario.
- **Evidencia Visual:**

```
[ERROR] COMPILATION ERROR :  
[ERROR] src/test/java/com/dbodor/veterinariaweb/admin/HU16CrearServicioTest.java:[3,39] cannot find symbol  
symbol:   class EstadoServicio  
location: package com.dbodor.veterinariaweb.model  
[ERROR] src/test/java/com/dbodor/veterinariaweb/admin/HU16CrearServicioTest.java:[5,44] cannot find symbol  
symbol:   class ServicioRepository  
location: package com.dbodor.veterinariaweb.repository  
[ERROR] src/test/java/com/dbodor/veterinariaweb/admin/HU16CrearServicioTest.java:[36,16] cannot find symbol  
symbol:   class ServicioRepository  
location: class com.dbodor.veterinariaweb.admin.HU16CrearServicioTest  
[ERROR] src/test/java/com/dbodor/veterinariaweb/admin/HU16CrearServicioTest.java:[43,47] cannot find symbol  
symbol:   class EstadoServicio  
location: class com.dbodor.veterinariaweb.admin.HU16CrearServicioTest  
[INFO] BUILD FAILURE
```

Extracto de evidencias\hu16\rojo\hu16-rojo.txt (rutas abreviadas).

2. Fase Verde (GREEN):

- **Descripción:** Se implementaron ServicioForm, con la validación de duración mediante @AssertTrue, el método crear() en ServicioServiceImpl, AdminServicioController y sus plantillas. Resultado de la suite: 57 pruebas, 0 fallos.
- **Evidencia Visual:**

```
Resumen de target/surefire-reports (hu16/verde/surefire-reports)  
  
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- in VeterinariaWebApplicationTests  
Tests run: 11, Failures: 0, Errors: 0, Skipped: 0 -- in HU01RegistrarClienteTest  
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0 -- in HU06RegistrarVeterinarioTest  
Tests run: 15, Failures: 0, Errors: 0, Skipped: 0 -- in HU16CrearServicioTest  
Tests run: 11, Failures: 0, Errors: 0, Skipped: 0 -- in HU02AutenticacionTest  
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 -- in HU21PanelAdministracionTest  
  
Total -> Tests run: 57, Failures: 0, Errors: 0, Skipped: 0
```

Resumen de los reportes de Surefire guardados en evidencias\hu16\verde\surefire-reports.

3. Fase de Refactorización (REFACTOR):

- **Descripción:** crear() delega en guardarServicio(), de modo que la regla de duración se aplica también cuando el servicio se guarda desde otro punto del sistema. Se documentó que los errores de @AssertTrue sobre un método se asocian a la propiedad derivada (duracionEnBloquesValidos), no al campo.

Historia de Usuario HU-22: Catálogo de medicamentos (pantallas del maestro)

- **Como:** Administrador
- **Quiero:** registrar, consultar y desactivar productos desde una pantalla del panel
- **Para:** mantener el catálogo que usan los veterinarios en los tratamientos

Clase de prueba: HU22MaestroProductosTest.java (13 pruebas). Complementa las pruebas unitarias de ProductoServiceTest descritas al inicio de esta sección: aquellas verifican las reglas en el servicio y estas verifican los mismos criterios de punta a punta. Prueba de integración con @SpringBootTest y MockMvc sobre H2 en memoria (perfil test). Todas las pruebas de la historia se escribieron antes del código de producción y se ejecutaron juntas en cada fase.

Criterio de Aceptación 1:

Dado que el administrador diligencia nombre, presentación, precio y stock, **cuando** guarda desde el formulario, **entonces** el producto queda registrado en estado ACTIVO con todos sus datos.

Criterio de Aceptación 2:

Dado que el precio es menor o igual a cero o el stock es negativo, **cuando** intenta guardar, **entonces** el formulario señala el campo erróneo y no persiste nada; un stock igual a cero sí se acepta.

Criterio de Aceptación 3:

Dado que un producto ya no se usa, **cuando** el administrador presiona Desactivar, **entonces** el producto pasa a INACTIVO y sigue apareciendo en el listado.

Trazabilidad entre criterios y pruebas:

Criterio	Pruebas (@DisplayName)
CA1	un producto valido se guarda y redirige al listado; el producto queda ACTIVO y con todos sus datos; un campo obligatorio vacio impide guardar y no persiste nada
CA2	un precio igual a cero se rechaza; un precio negativo se rechaza; un stock negativo se rechaza; un stock igual a cero se acepta
CA3	desactivar deja el producto INACTIVO sin eliminarlo; el producto desactivado sigue apareciendo en el listado como INACTIVO
Listado	muestra los productos con nombre, presentacion, stock y estado; la busqueda filtra por nombre
Acceso	un cliente no entra al maestro de productos; sin sesion redirige al login

A. Código Fuente Involucrado

Código de la Prueba (HU22MaestroProductosTest.java):

```
@Test
@WithMockUser(roles = "ADMINISTRADOR")
@DisplayName("CA3 - desactivar deja el producto INACTIVO sin eliminarlo")
void desactivarDejaInactivo() throws Exception {
    Producto p = producto("Amoxicilina", "Tabletas 250 mg", EstadoProducto.ACTIVO);

    mockMvc.perform(post("/admin/productos/" + p.getIdProducto() + "/desactivar").with(csrf()))
        .andExpect(status().is3xxRedirection())
        .andExpect(redirectedUrl("/admin/productos"));

    assertThat(productoRepository.count()).isEqualTo(1);
    assertThat(productoRepository.findById(p.getIdProducto()).orElseThrow().getEstado())
        .isEqualTo(EstadoProducto.INACTIVO);
}
```

Código de Producción Implementado (AdminProductoController.java):

```
@PostMapping("/{id}/desactivar")
public String desactivar(@PathVariable Long id, RedirectAttributes flash) {
    try {
        Producto producto = productoService.desactivarProducto(id);
        flash.addFlashAttribute("mensajeExito",
            "Producto " + producto.getNombre() + " desactivado.");
    } catch (IllegalArgumentException e) {
        flash.addFlashAttribute("mensajeError", e.getMessage());
    }
    return "redirect:/admin/productos";
}
```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** El servicio de productos existía, pero no había controlador ni pantallas, y ProductoServiceImpl no tenía la anotación @Service. Primera ejecución: 13 pruebas, 10 fallos y 1 error. El error fue un getFirst() sobre una lista vacía, porque ningún producto llegaba a guardarse.
- **Evidencia Visual:**

```
[ERROR] Tests run: 13, Failures: 10, Errors: 1, Skipped: 0, Time elapsed: 10.41 s <<< FAILURE! -- in com.dbod...
[INFO] Results:
[ERROR] HU22MaestroProductosTest.busquedaPorNombre:212 No ModelAndView found
[ERROR] HU22MaestroProductosTest.desactivadoSigueEnListado:183 Response content
[ERROR] HU22MaestroProductosTest.desactivarDejaInactivo:168 Range for response status value 404 expected:<R...
[ERROR] HU22MaestroProductosTest.listadoMuestraSusDatos:196 Status expected:<200> but was:<404>
[ERROR] HU22MaestroProductosTest.nombreVacioSeRechaza:107 Status expected:<200> but was:<404>
[ERROR] HU22MaestroProductosTest.precioCeroSeRechaza:121 Status expected:<200> but was:<404>
[ERROR] HU22MaestroProductosTest.precioNegativoSeRechaza:132 Status expected:<200> but was:<404>
[ERROR] HU22MaestroProductosTest.productoValidoSeGuarda:81 Range for response status value 404 expected:<RE...
[ERROR] ... (3 más)
[ERROR] Tests run: 13, Failures: 10, Errors: 1, Skipped: 0
[INFO] BUILD FAILURE
[INFO] Total time: 18.579 s
[INFO] Finished at: 2026-09-26T01:18:37-05:00
```

Extracto de evidencias\HU22-pantallas\rojo\consola.txt (salida de mvnw test).

2. Fase Verde (GREEN):

- **Descripción:** Se agregaron @Service y un método listar() a ProductoServiceImpl sin modificar su lógica, una consulta de búsqueda en ProductoRepository, ProductoForm, AdminProductoController y sus plantillas. El primer intento de fase verde no compiló porque ProductoRepository no se había reemplazado; corregido eso, el resultado fue 13 pruebas, 0 fallos, y la suite completa quedó en 127 pruebas en verde, incluidas las 4 pruebas unitarias originales de ProductoServiceTest.
- **Evidencia Visual:**

```
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 11.43 s -- in com.dbod.veterinariaw...
[INFO] Results:
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 27.630 s
[INFO] Finished at: 2026-09-26T01:28:55-05:00
```

Extracto de evidencias\HU22-pantallas\verde\consola.txt (salida de mvnw test).

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Se mantuvo la validación de precio y stock en el servicio y se repitió en ProductoForm, para que el error aparezca junto al campo. Después de la entrega se detectó en uso real que el campo de precio tenía step="100" y el navegador bloqueaba precios válidos; las pruebas no lo detectaron porque MockMvc no ejecuta la validación HTML del navegador (ver sección 4).

Épica: Gestión de Mascotas

Historia de Usuario HU-11: Registrar una mascota

- **Como:** Administrador

- **Quiero:** registrar las mascotas de cada cliente con sus datos básicos
- **Para:** poder agendarles citas y llevar su historia

Clase de prueba: HU11RegistrarMascotaTest.java (15 pruebas). La historia original estaba escrita para el rol cliente; se adaptó al maestro del administrador. Prueba de integración con @SpringBootTest y MockMvc sobre H2 en memoria (perfil test). Todas las pruebas de la historia se escribieron antes del código de producción y se ejecutaron juntas en cada fase.

Criterio de Aceptación 1:

Dado que diligencio el dueño, nombre, especie, raza, sexo, fecha de nacimiento y peso, **cuando** guardo, **entonces** la mascota queda registrada, activa y asociada al cliente.

Criterio de Aceptación 2:

Dado que el dueño enviado no es un cliente activo, o no se indicó, **cuando** intento guardar, **entonces** el sistema rechaza el registro aunque el identificador se haya enviado manipulado.

Criterio de Aceptación 3:

Dado que la especie no es perro, gato, ave, roedor u otro, **cuando** intento guardar, **entonces** el sistema la rechaza.

Criterio de Aceptación 4:

Dado que la fecha de nacimiento es futura, **cuando** intento guardar, **entonces** el sistema la rechaza; la fecha de hoy se acepta.

Criterio de Aceptación 5:

Dado que el peso es cero o negativo, **cuando** intento guardar, **entonces** el sistema lo rechaza.

Trazabilidad entre criterios y pruebas:

Criterio	Pruebas (@DisplayName)
CA1	una mascota valida se guarda y redirige al listado; la mascota queda asociada al cliente, activa y con todos sus datos; un campo obligatorio vacio impide guardar y no persiste nada
CA2	el formulario ofrece como dueno solo clientes activos; un dueno que no es cliente se rechaza aunque se envíe a mano; sin dueno no se puede registrar

Criterio	Pruebas (@DisplayName)
CA3	el formulario ofrece solo perro, gato, ave, roedor u otro; una especie fuera de la lista se rechaza
CA4	una fecha de nacimiento futura se rechaza; una mascota nacida hoy se acepta
CA5	un peso igual a cero se rechaza; un peso negativo se rechaza
Regla	un mismo dueño no puede tener dos mascotas con el mismo nombre
Acceso	un cliente no entra al alta de mascotas del administrador; sin sesión redirige al login

A. Código Fuente Involucrado

Código de la Prueba (HU11RegistrarMascotaTest.java):

```

@Test
@WithMockUser(roles = "ADMINISTRADOR")
@DisplayName("Regla - un mismo dueño no puede tener dos mascotas con el mismo nombre")
void nombreRepetidoParaElMismoDueñoSeRechaza() throws Exception {
    registrarValida("Firulais");

    registrarValida("firulais")
        .andExpect(status().isOk())
        .andExpect(model().attributeHasFieldErrors("mascotaForm", "nombre"));

    assertThat(mascotaRepository.count()).isEqualTo(1);
}

```

Código de Producción Implementado (MascotaServiceImpl.java):

```

@Override
@Transactional
public Mascota registrar(MascotaForm form) {
    // CA2: el formulario solo ofrece clientes activos, pero el id puede
    // llegar manipulado, así que se vuelve a comprobar aquí.
    Usuario dueño = usuarioRepository.findById(form.getIdCliente())
        .filter(u -> u.getRol() == RolUsuario.CLIENTE)
        .filter(u -> u.getEstado() == EstadoUsuario.ACTIVO)
        .orElseThrow(() -> new IllegalArgumentException("El dueño debe ser un cliente activo"));

    String nombre = form.getNombre().trim();
    if (mascotaRepository.existsByUsuarioIdUsuarioAndNombreIgnoreCase(dueño.getIdUsuario(),
        nombre)) {
        throw new DatoDuplicadoException("nombre", "Este cliente ya tiene una mascota con ese nombre");
    }

    Mascota mascota = new Mascota();
}

```

```

mascota.setUsuario(dueno);
mascota.setNombre(nombre);
mascota.setEspecie(form.getEspecie());
mascota.setRaza(form.getRaza() == null || form.getRaza().isBlank() ? null : form.getRaza().trim());
mascota.setSexo(form.getSexo());
mascota.setFechaNacimiento(form.getFechaNacimiento());
mascota.setPesoKg(form.getPesoKg());
mascota.setEstado(ESTADO_ACTIVO);

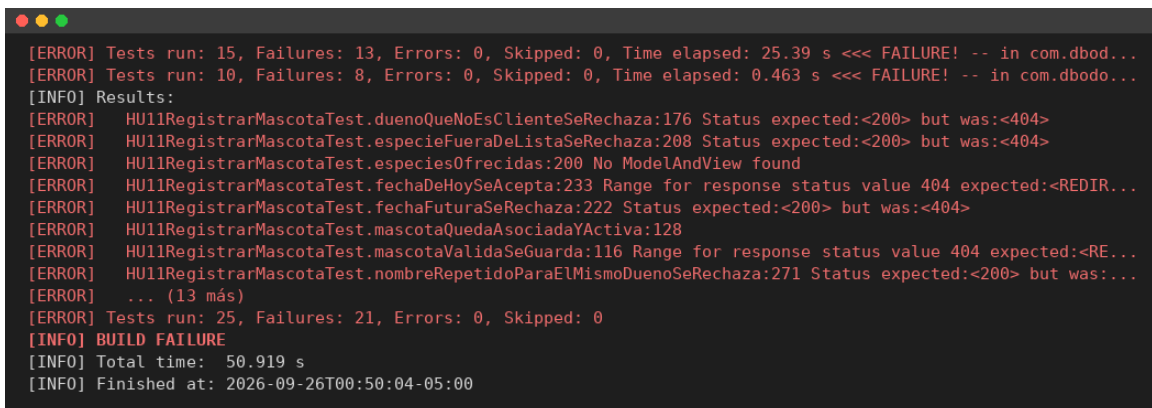
return mascotaRepository.save(mascota);
}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** HU-11 y HU-12 se ejecutaron juntas en la fase roja: 25 pruebas, 21 fallos. Las 4 que pasaron son las de acceso de ambas clases. Se agrega una regla de modelo, nombre único por dueño, que ya estaba prevista en MascotaRepository.
- **Evidencia Visual:**



```

[ERROR] Tests run: 15, Failures: 13, Errors: 0, Skipped: 0, Time elapsed: 25.39 s <<< FAILURE! -- in com.dbod...
[ERROR] Tests run: 10, Failures: 8, Errors: 0, Skipped: 0, Time elapsed: 0.463 s <<< FAILURE! -- in com.dbodo...
[INFO] Results:
[ERROR]   HU11RegistrarMascotaTest.duenoQueNoEsClienteSeRechaza:176 Status expected:<200> but was:<404>
[ERROR]   HU11RegistrarMascotaTest.especieFueraDeListaSeRechaza:208 Status expected:<200> but was:<404>
[ERROR]   HU11RegistrarMascotaTest.especiesOfrecidas:200 No ModelAndView found
[ERROR]   HU11RegistrarMascotaTest.fechaDeHoySeAcepta:233 Range for response status value 404 expected:<REDIR...
[ERROR]   HU11RegistrarMascotaTest.fechaFuturaSeRechaza:222 Status expected:<200> but was:<404>
[ERROR]   HU11RegistrarMascotaTest.mascotaQuedaAsociadaYActiva:128
[ERROR]   HU11RegistrarMascotaTest.mascotaValidaSeGuarda:116 Range for response status value 404 expected:<RE...
[ERROR]   HU11RegistrarMascotaTest.nombreRepetidoParaElMismoDuenoSeRechaza:271 Status expected:<200> but was:...
[ERROR]   ... (13 más)
[ERROR] Tests run: 25, Failures: 21, Errors: 0, Skipped: 0
[INFO] BUILD FAILURE
[INFO] Total time: 50.919 s
[INFO] Finished at: 2026-09-26T00:50:04-05:00

```

Extracto de evidencias\HU11-HU12\rojo\consola.txt (salida de mvnw test).

2. Fase Verde (GREEN):

- **Descripción:** Se implementaron MascotaForm con @PastOrPresent, @Positive y @Pattern para la especie, MascotaService, MascotaServiceImpl, AdminMascotaController y sus plantillas. Resultado: 25 pruebas, 0 fallos; suite completa, 114 pruebas en verde.
- **Evidencia Visual:**

```
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 21.75 s -- in com.dbodor.veterinariaw...
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.399 s -- in com.dbodor.veterinariaw...
[INFO] Results:
[INFO] Tests run: 25, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 41.912 s
[INFO] Finished at: 2026-09-26T00:57:17-05:00
```

Extracto de evidencias\HU11-HU12\verde\consola.txt (salida de mvnw test).

3. Fase de Refactorización (REFACTOR):

- **Descripción:** Las reglas de formato quedaron en el formulario y las reglas que requieren consultar la base (dueño válido y nombre repetido) en el servicio, que las revalida aunque el formulario solo ofrezca opciones válidas.

Historia de Usuario HU-12: Consultar las mascotas registradas

- **Como:** Administrador
- **Quiero:** consultar las mascotas registradas con sus datos y su edad
- **Para:** ubicar rápidamente una mascota y a su dueño

Clase de prueba: HU12ConsultarMascotasTest.java (10 pruebas). Prueba de integración con @SpringBootTest y MockMvc sobre H2 en memoria (perfil test). Todas las pruebas de la historia se escribieron antes del código de producción y se ejecutaron juntas en cada fase.

Criterio de Aceptación 1:

Dado que existen mascotas registradas, **cuando** abro el maestro, **entonces** veo todas y puedo buscarlas por el nombre de la mascota, del dueño o por su documento.

Criterio de Aceptación 2:

Dado que consulto el listado, **cuando** reviso cada fila, **entonces** veo nombre, especie, raza, dueño, estado y la edad calculada, en años o en meses si es menor de un año.

Criterio de Aceptación 3:

Dado que no hay mascotas registradas, **cuando** abro el maestro, **entonces** el sistema me invita a registrar la primera.

Criterio de Aceptación 4:

Dado que soy cliente o no tengo sesión, **cuando** intento abrir el maestro, **entonces** el sistema me redirige a mi inicio o al login.

Trazabilidad entre criterios y pruebas:

Criterio	Pruebas (@DisplayName)
CA1	el listado muestra todas las mascotas registradas; la búsqueda filtra por el nombre de la mascota; la búsqueda filtra por el nombre del dueño; la búsqueda filtra por el documento del dueño
CA2	cada fila muestra nombre, especie, raza, dueño y estado; la edad se calcula en años a partir de la fecha de nacimiento; una mascota de menos de un año muestra la edad en meses
CA3	sin mascotas el sistema invita a registrar la primera
CA4	un cliente no entra al maestro de mascotas; sin sesión redirige al login

A. Código Fuente Involucrado

Código de la Prueba (HU12ConsultarMascotasTest.java):

```
@Test
@WithMockUser(roles = "ADMINISTRADOR")
@DisplayName("CA2 - una mascota de menos de un año muestra la edad en meses")
void edadEnMeses() throws Exception {
    mockMvc.perform(get("/admin/mascotas"))
        .andExpect(content().string(containsString("5 meses")));
}
```

Código de Producción Implementado (Mascota.java):

```
public String getEdad() {
    if (fechaNacimiento == null) {
        return "Sin dato";
    }
    Period p = Period.between(fechaNacimiento, LocalDate.now());
    if (p.getYears() >= 1) {
        return p.getYears() + (p.getYears() == 1 ? " año" : " años");
    }
    if (p.getMonths() >= 1) {
        return p.getMonths() + (p.getMonths() == 1 ? " mes" : " meses");
    }
    return "Menos de un mes";
}
```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Ejecutada junto con HU-11 (ver la historia anterior).
- **Evidencia Visual:**

Evidencia: la misma ejecución mostrada en la fase roja de HU-11.

2. Fase Verde (GREEN):

- **Descripción:** Se agregó la consulta buscar() a MascotaRepository, con join fetch del dueño, y el método getEdad() en la entidad. Resultado conjunto: 25 pruebas, 0 fallos.
- **Evidencia Visual:**

Evidencia: la misma ejecución mostrada en la fase verde de HU-11.

3. Fase de Refactorización (REFACTOR):

- **Descripción:** La edad no se persiste: se calcula en cada consulta para que nunca quede desactualizada. El join fetch fue necesario porque el proyecto desactiva open-in-view y la plantilla no puede cargar relaciones perezosas.

Épica: Gestión de Citas

Historia de Usuario HU-25: Agendar una cita

- **Como:** Administrador
- **Quiero:** agendar citas eligiendo cliente, mascota, servicio, veterinario, fecha y hora
- **Para:** organizar la agenda de la clínica sin cruces de horario

Clase de prueba: HU25AgendarCitaTest.java (14 pruebas). Es la pantalla transaccional principal del sistema. Prueba de integración con @SpringBootTest y MockMvc sobre H2 en memoria (perfil test). Todas las pruebas de la historia se escribieron antes del código de producción y se ejecutaron juntas en cada fase.

Criterio de Aceptación 1:

Dado que abro el formulario de agendamiento, **cuando** reviso las opciones, **entonces** solo se ofrecen servicios y veterinarios activos, y un servicio inactivo enviado a mano se rechaza.

Criterio de Aceptación 2:

Dado que elijo un cliente, **cuando** la pantalla se actualiza, **entonces** solo se ofrecen sus mascotas activas.

Criterio de Aceptación 3:

Dado que los datos son válidos, **cuando** agendo, **entonces** la cita queda PROGRAMADA, con fecha de creación y con valor igual al precio del servicio más la tarifa del veterinario.

Criterio de Aceptación 4:

Dado que el veterinario ya tiene una cita que se cruza con el horario, **cuando** agendo, **entonces** el sistema la rechaza; una cita que empieza justo cuando termina la anterior sí se acepta.

Criterio de Aceptación 5:

Dado que la fecha y hora ya pasaron, **cuando** agendo, **entonces** el sistema la rechaza.

Criterio de Aceptación 6:

Dado que la cita empieza antes o termina después de la jornada del veterinario, **cuando** agendo, **entonces** el sistema la rechaza.

Criterio de Aceptación 7:

Dado que la cita fue agendada, **cuando** abro el listado, **entonces** aparece con mascota, servicio, veterinario y estado.

Trazabilidad entre criterios y pruebas:

Criterio	Pruebas (@DisplayName)
CA1	el formulario ofrece solo servicios activos; el formulario ofrece solo veterinarios activos; un servicio inactivo enviado a mano no se puede agendar
CA2	al elegir un cliente se ofrecen solo sus mascotas activas
CA3	una cita valida se guarda y redirige al listado; la cita queda PROGRAMADA, con fecha de creacion y valor = precio del servicio + tarifa
CA4	un servicio de 60 min a las 09:00 bloquea al veterinario hasta las 10:00; la franja que empieza justo cuando termina la anterior se acepta
CA5	una fecha ya pasada se rechaza

Criterio	Pruebas (@DisplayName)
CA6	una hora anterior al inicio de la jornada se rechaza; una cita que termina despues del fin de la jornada se rechaza
CA7	la cita agendada aparece en el listado con mascota, servicio, veterinario y estado
Acceso	un cliente no entra a la pantalla de reservas del administrador; sin sesion redirige al login

A. Código Fuente Involucrado

Código de la Prueba (HU25AgendarCitaTest.java):

```
@Test
@WithMockUser(roles = "ADMINISTRADOR")
@DisplayName("CA4 - un servicio de 60 min a las 09:00 bloquea al veterinario hasta las 10:00")
void franjaSolapadaSeRechaza() throws Exception {
    agendar(firulais, consulta, doctora, fechaFutura, "09:00");

    agendar(firulais, consulta, doctora, fechaFutura, "09:30")
        .andExpect(status().isOk())
        .andExpect(view().name("admin/citas/formulario"))
        .andExpect(content().string(containsString("El veterinario ya tiene una cita asignada en ese rango horario"))));

    assertThat(citaRepository.count()).isEqualTo(1);
}
```

Código de la Prueba (HU25AgendarCitaTest.java):

```
@Test
@WithMockUser(roles = "ADMINISTRADOR")
@DisplayName("CA3 - la cita queda PROGRAMADA, con fecha de creacion y valor = precio del servicio + tarifa")
void citaQuedaProgramadaConValor() throws Exception {
    LocalDateTime antes = LocalDateTime.now().minusSeconds(1);

    agendar(firulais, consulta, doctora, fechaFutura, "09:00");

    Cita guardada = citaRepository.findAll().getFirst();
    assertThat(guardada.getEstado()).isEqualTo("PROGRAMADA");
    assertThat(guardada.getFechaCita()).isEqualTo(fechaFutura);
    assertThat(guardada.getHoraCita()).isEqualTo(LocalTime.of(9, 0));
    assertThat(guardada.getCreatedAt()).isAfter(antes);
    assertThat(guardada.getCostoTotal()).isEqualTo(100000.0);
}
```

Código de Producción Implementado (AgendaCitaServiceImpl.java):

```
@Override
@Transactional
public Cita agendar(CitaForm form) {
    Mascota mascota = mascotaRepository.findById(form.getIdMascota())
        .orElseThrow(() -> new IllegalArgumentException("La mascota seleccionada no existe"));
    Servicio servicio = servicioRepository.findById(form.getIdServicio())
        .orElseThrow(() -> new IllegalArgumentException("El servicio seleccionada no existe"));
    Veterinario veterinario = veterinarioRepository.findById(form.getIdVeterinario())
        .orElseThrow(() -> new IllegalArgumentException("El veterinario seleccionado no existe"));

    // CA1 y CA2: aunque el formulario solo ofrece opciones validas, los
    // identificadores pueden llegar manipulados, así que se revalidan.
    if (servicio.getEstado() != EstadoServicio.ACTIVO) {
        throw new IllegalArgumentException("El servicio seleccionado no esta disponible");
    }
    if (!veterinario.estaActivo()) {
        throw new IllegalArgumentException("El veterinario seleccionado no esta disponible");
    }
    if (!MASCOTA_ACTIVA.equals(mascota.getEstado())) {
        throw new IllegalArgumentException("La mascota seleccionada no esta activa");
    }

    // CA5: no se agenda en el pasado.
    if (LocalDateTime.of(form.getFecha(), form.getHora()).isBefore(LocalDateTime.now())) {
        throw new IllegalArgumentException("La fecha y hora de la cita ya pasaron");
    }

    // CA6: la cita completa debe caber en la jornada del veterinario.
    LocalTime inicio = form.getHora();
    LocalTime fin = inicio.plusMinutes(servicio.getDuracionMinutos());
    boolean cruzaMedianoche = fin.isBefore(inicio);
    if (inicio.isBefore(veterinario.getHoraInicio())
        || fin.isAfter(veterinario.getHoraFin())
        || cruzaMedianoche) {
        throw new IllegalArgumentException("La cita queda fuera del horario del veterinario");
    }

    // CA3: el valor se congela al agendar, para que un cambio posterior
    // de precio o tarifa no altere las citas ya registradas (HU-17).
    double valor = servicio.getPrecioBase()
        + (veterinario.getTarifa() == null ? 0.0 : veterinario.getTarifa().doubleValue());

    Cita cita = new Cita();
    cita.setMascota(mascota);
    cita.setServicio(servicio);
    cita.setVeterinario(veterinario);
    cita.setFechaCita(form.getFecha());
    cita.setHoraCita(inicio);
    cita.setCostoTotal(valor);
    cita.setEstado(ESTADO_PROGRAMADA);
    cita.setCreatedAt(LocalDateTime.now());

    // CA4: el solapamiento lo controla CitaService y lanza IllegalStateException.
```

```

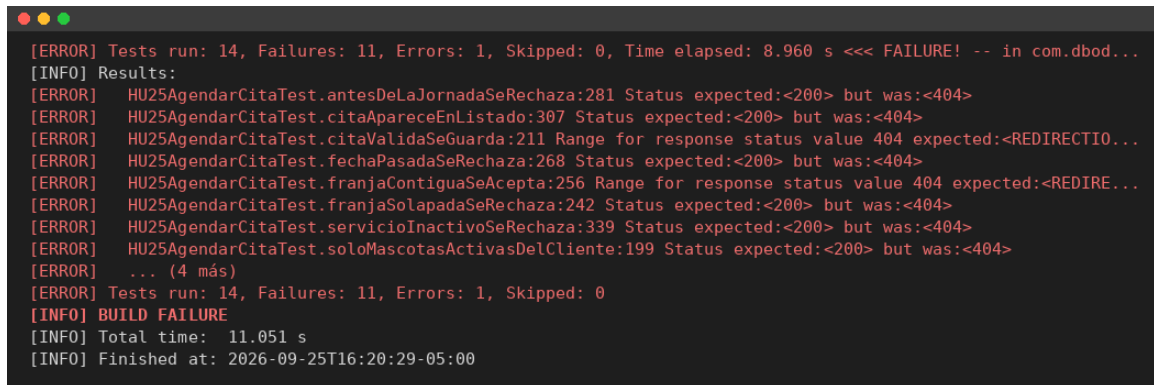
return citaService.agendarCita(cita);
}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Fase roja: 14 pruebas, 11 fallos y 1 error. El error fue un `getFirst()` sobre una lista vacía en la prueba del valor de la cita, porque ninguna cita llegaba a guardarse. Las 2 pruebas de acceso pasaron.
- **Evidencia Visual:**



```

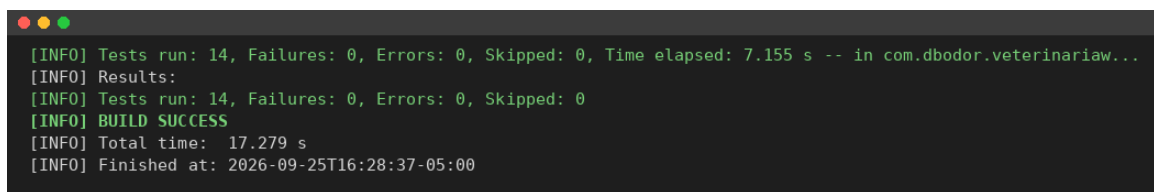
[ERROR] Tests run: 14, Failures: 11, Errors: 1, Skipped: 0, Time elapsed: 8.960 s <<< FAILURE! -- in com.dbod...
[INFO] Results:
[ERROR] HU25AgendarCitaTest.antesDeLaJornadaSeRechaza:281 Status expected:<200> but was:<404>
[ERROR] HU25AgendarCitaTest.citaApareceEnListado:307 Status expected:<200> but was:<404>
[ERROR] HU25AgendarCitaTest.citaValidaSeGuarda:211 Range for response status value 404 expected:<REDIRECTIO...
[ERROR] HU25AgendarCitaTest.fechaPasadaSeRechaza:268 Status expected:<200> but was:<404>
[ERROR] HU25AgendarCitaTest.franjaContiguaSeAcepta:256 Range for response status value 404 expected:<REDIRE...
[ERROR] HU25AgendarCitaTest.franjaSolapadaSeRechaza:242 Status expected:<200> but was:<404>
[ERROR] HU25AgendarCitaTest.servicioInactivoSeRechaza:339 Status expected:<200> but was:<404>
[ERROR] HU25AgendarCitaTest.soloMascotasActivasDelCliente:199 Status expected:<200> but was:<404>
[ERROR] ... (4 más)
[ERROR] Tests run: 14, Failures: 11, Errors: 1, Skipped: 0
[INFO] BUILD FAILURE
[INFO] Total time: 11.051 s
[INFO] Finished at: 2026-09-25T16:20:29-05:00

```

Extracto de evidencias\HU25\rojo\consola.txt (salida de mvnw test).

2. Fase Verde (GREEN):

- **Descripción:** Se implementaron `CitaForm`, `AgendaCitaService`, `AgendaCitaServiceImpl`, `AdminCitaController` y dos plantillas; se agregó `@Service` a `CitaServiceImpl`, que no la tenía, y una consulta con `join fetch` en `CitaRepository`. Resultado: 14 pruebas, 0 fallos; suite completa, 89 pruebas en verde.
- **Evidencia Visual:**



```

[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 7.155 s -- in com.dbodor.veterinariaw...
[INFO] Results:
[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 17.279 s
[INFO] Finished at: 2026-09-25T16:28:37-05:00

```

Extracto de evidencias\HU25\verde\consola.txt (salida de mvnw test).

3. Fase de Refactorización (REFACTOR):

- **Descripción:** El control de solapamiento no se duplicó: `AgendaCitaServiceImpl` delega en `CitaServiceImpl.agendarCita()`, que ya lo resolvía y tenía su propia prueba unitaria (HU-20 CA3). El valor se congela al agendar para que un cambio posterior de precio o tarifa no altere

las citas registradas. Se unificó el estado de las citas en PROGRAMADA (ver sección 4).

Épica: Reportes

Historia de Usuario HU-26: Reporte de citas e ingresos por período

- **Como:** Administrador
- **Quiero:** generar un reporte de citas e ingresos filtrado por período y veterinario
- **Para:** conocer la operación y la facturación de la clínica

Clase de prueba: HU26ReporteCitasTest.java (13 pruebas). Las citas se insertan con fechas fijas de agosto y septiembre de 2026 para que el resultado no dependa del día de ejecución. Prueba de integración con @SpringBootTest y MockMvc sobre H2 en memoria (perfil test). Todas las pruebas de la historia se escribieron antes del código de producción y se ejecutaron juntas en cada fase.

Criterio de Aceptación 1:

Dado que selecciono una fecha inicial y una final, **cuando** genero el reporte, **entonces** el sistema lista las citas del rango, ambas fechas inclusive, con fecha, cliente, mascota, servicio, veterinario, estado y valor.

Criterio de Aceptación 2:

Dado que aplico el filtro por veterinario, **cuando** genero el reporte, **entonces** solo se incluyen sus citas.

Criterio de Aceptación 3:

Dado que genero el reporte, **cuando** reviso el resumen, **entonces** veo el total de citas, las finalizadas, las canceladas y la suma de ingresos de las finalizadas, también con el filtro aplicado.

Criterio de Aceptación 4:

Dado que un veterinario o un servicio fue desactivado, **cuando** genero un reporte de un período anterior, **entonces** sus citas siguen apareciendo y el filtro ofrece también a los veterinarios inactivos.

Criterio de Aceptación 5:

Dado que la fecha inicial es posterior a la final, **cuando** genero el reporte, **entonces** el sistema rechaza el rango y explica el error.

Criterio de Aceptación 6:

Dado que no hay citas en el rango, **cuando** genero el reporte, **entonces** el sistema informa que no hay datos para ese período.

Trazabilidad entre criterios y pruebas:

Criterio	Pruebas (@DisplayName)
Pantalla	sin fechas muestra el formulario y todavia no genera reporte
CA1	el reporte trae solo las citas dentro del rango; las dos fechas del rango son inclusivas; cada fila muestra fecha, cliente, mascota, servicio, veterinario, estado y valor
CA2	el filtro por veterinario deja solo sus citas
CA3	el resumen cuenta total, finalizadas, canceladas e ingresos de las finalizadas; el resumen respeta el filtro por veterinario
CA4	las citas de un veterinario y un servicio desactivados siguen en el historico; el filtro ofrece tambien a los veterinarios inactivos
CA5	una fecha inicial posterior a la final se rechaza y se explica
CA6	un periodo sin citas informa que no hay datos
Acceso	un cliente no entra al reporte; sin sesion redirige al login

A. Código Fuente Involucrado

Código de la Prueba (HU26ReporteCitasTest.java):

```
@Test
@WithMockUser(roles = "ADMINISTRADOR")
@DisplayName("CA3 - el resumen cuenta total, finalizadas, canceladas e ingresos de las finalizadas")
void resumenDelPeriodo() throws Exception {
    generar("2026-08-01", "2026-08-31")
        .andExpect(model().attribute("reporte", hasProperty("total", is(4L))))
        .andExpect(model().attribute("reporte", hasProperty("finalizadas", is(2L))))
        .andExpect(model().attribute("reporte", hasProperty("canceladas", is(1L))))
        .andExpect(model().attribute("reporte", hasProperty("ingresos", is(180000.0))));
}
```

Código de Producción Implementado (ReporteServiceImpl.java):

```
@Override
@Transactional(readOnly = true)
```

```

public ReporteCitas citasElIngresos(LocalDate desde, LocalDate hasta, Long idVeterinario) {
    if (desde.isAfter(hasta)) {
        throw new IllegalArgumentException("La fecha inicial no puede ser posterior a la final");
    }

    // CA4: la consulta no filtra por estado del veterinario ni del
    // servicio, así que las citas de los desactivados siguen apareciendo.
    List<Cita> citas = citaRepository.reportePorRango(desde, hasta).stream()
        .filter(c -> idVeterinario == null
            || idVeterinario.equals(c.getVeterinario().getIdVeterinario()))
        .toList();

    long finalizadas = citas.stream().filter(c -> ESTADO_FINALIZADA.equals(c.getEstado())).count();
    long canceladas = citas.stream().filter(c -> ESTADO_CANCELADA.equals(c.getEstado())).count();
    double ingresos = citas.stream()
        .filter(c -> ESTADO_FINALIZADA.equals(c.getEstado()))
        .mapToDouble(c -> c.getCostoTotal() == null ? 0.0 : c.getCostoTotal())
        .sum();

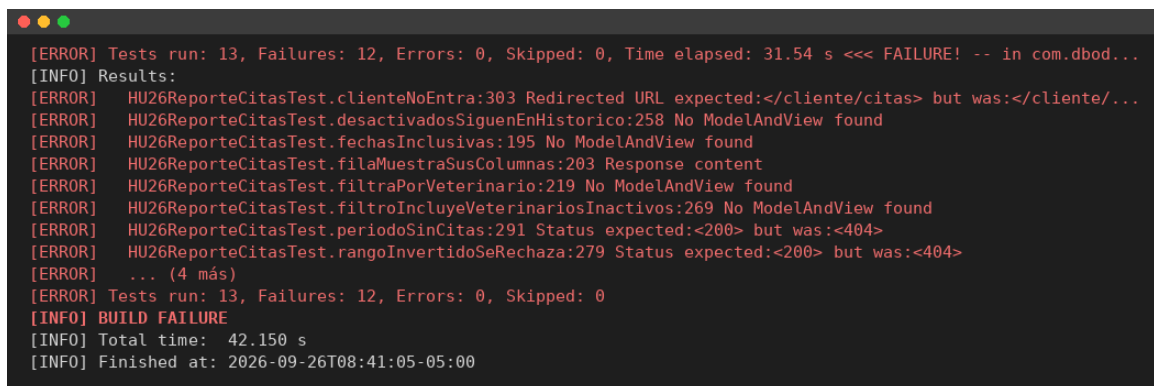
    return new ReporteCitas(citas, citas.size(), finalizadas, canceladas, ingresos);
}

```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- Descripción:** Se ejecutó la clase antes de crear el reporte: 13 pruebas, 12 fallos. Solo pasó la prueba de usuario sin sesión. Uno de los 12 fallos no se debía a la ausencia del reporte sino a la prueba de acceso del cliente, que esperaba `/cliente/citas` cuando el sistema ya redirigía a `/cliente/dashboard`. Antes de escribir las pruebas se decidió que una cita completada es la que está en estado FINALIZADA, el único que el sistema asigna (ver sección 4).
- Evidencia Visual:**



```

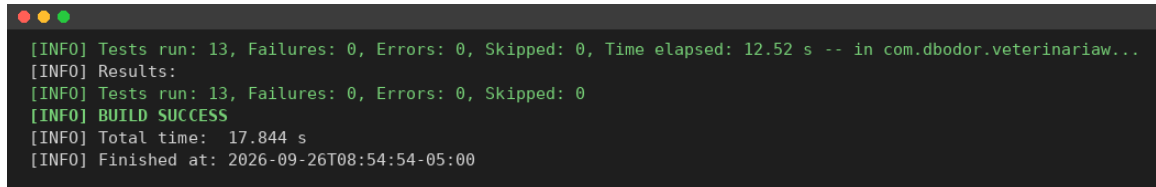
[ERROR] Tests run: 13, Failures: 12, Errors: 0, Skipped: 0, Time elapsed: 31.54 s <<< FAILURE! -- in com.dbod...
[INFO] Results:
[ERROR] HU26ReporteCitasTest.clienteNoEntra:303 Redirected URL expected:</cliente/citas> but was:</cliente/...
[ERROR] HU26ReporteCitasTest.desactivadosSiguenEnHistorico:258 No ModelAndView found
[ERROR] HU26ReporteCitasTest.fechaInclusivas:195 No ModelAndView found
[ERROR] HU26ReporteCitasTest.filaMuestraSusColumnas:203 Response content
[ERROR] HU26ReporteCitasTest.filtraPorVeterinario:219 No ModelAndView found
[ERROR] HU26ReporteCitasTest.filtroIncluyeVeterinariosInactivos:269 No ModelAndView found
[ERROR] HU26ReporteCitasTest.periodoSinCitas:291 Status expected:<200> but was:<404>
[ERROR] HU26ReporteCitasTest.rangoInvertidoSeRechaza:279 Status expected:<200> but was:<404>
[ERROR] ... (4 más)
[ERROR] Tests run: 13, Failures: 12, Errors: 0, Skipped: 0
[INFO] BUILD FAILURE
[INFO] Total time: 42.150 s
[INFO] Finished at: 2026-09-26T08:41:05-05:00

```

Extracto de evidencias\HU26\rojo\consola.txt (salida de mvnw test).

2. Fase Verde (GREEN):

- **Descripción:** Se implementaron ReporteCitas, ReporteService, ReporteServiceImpl, AdminReporteController y la plantilla del reporte. La primera ejecución de la fase verde dio 13 pruebas y 1 fallo: el mismo fallo de la prueba de acceso del cliente, causado porque el panel del cliente había cambiado su ruta de inicio. Actualizada la aserción, el resultado fue 13 pruebas, 0 fallos.
- **Evidencia Visual:**



```
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 12.52 s -- in com.dbodor.veterinariaw...
[INFO] Results:
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 17.844 s
[INFO] Finished at: 2026-09-26T08:54:54-05:00
```

Extracto de evidencias\HU26\verdelconsola.txt (salida de mvnw test).

3. Fase de Refactorización (REFACTOR):

- **Descripción:** El filtro por veterinario se aplica en memoria sobre las citas del rango, en lugar de una condición opcional en la consulta, porque los parámetros nulos en ese tipo de condición suelen fallar en PostgreSQL y el volumen de datos no justifica la complejidad.

Historia de Usuario HU-24: Historia clínica en PDF

- **Como:** Administrador
- **Quiero:** consultar la historia clínica de una mascota y descargarla en PDF
- **Para:** tener a mano sus datos y todas sus atenciones

Clase de prueba: HU24HistoriaClinicaTest.java (10 pruebas). Prueba de integración con @SpringBootTest y MockMvc sobre H2 en memoria (perfil test). Todas las pruebas de la historia se escribieron antes del código de producción y se ejecutaron juntas en cada fase.

Criterio de Aceptación 1:

Dado que abro la pantalla de historias clínicas, **cuando** busco una mascota, **entonces** el sistema lista las mascotas y filtra por mascota, dueño o documento.

Criterio de Aceptación 2:

Dado que elijo una mascota, **cuando** abro su historia, **entonces** veo sus datos, los de su dueño y sus atenciones con fecha, servicio, veterinario y estado.

Criterio de Aceptación 3:

Dado que la historia se muestra, **cuando** reviso las atenciones, **entonces** solo aparecen las de esa mascota.

Criterio de Aceptación 4:

Dado que la mascota no tiene atenciones, **cuando** abro su historia, **entonces** el sistema lo informa.

Criterio de Aceptación 5:

Dado que presiono Descargar PDF, **cuando** el sistema genera el documento, **entonces** se descarga un PDF real con nombre de archivo propio.

Criterio de Aceptación 6:

Dado que se solicita una mascota inexistente, **cuando** se abre la ruta, **entonces** el sistema vuelve al listado.

Trazabilidad entre criterios y pruebas:

Criterio	Pruebas (@DisplayName)
CA1	la pantalla lista las mascotas para elegir la historia; la búsqueda filtra por mascota, dueño o documento
CA2	la historia muestra los datos de la mascota y de su dueño; la historia lista las atenciones con fecha, servicio, veterinario y estado
CA3	solo aparecen las atenciones de la mascota consultada
CA4	una mascota sin atenciones lo informa
CA5	la descarga genera un documento PDF con nombre de archivo
CA6	una mascota inexistente vuelve al listado
Acceso	un cliente no entra a las historias clinicas del administrador; sin sesion redirige al login

A. Código Fuente Involucrado

Código de la Prueba (HU24HistoriaClinicaTest.java):

```
@Test
@WithMockUser(roles = "ADMINISTRADOR")
@DisplayName("CA5 - la descarga genera un documento PDF con nombre de archivo")
void descargaPdf() throws Exception {
    MvcResult resultado = mockMvc.perform(get("/admin/reportes/historias/" + firulais.getIdMascota() +
"/pdf"))
        .andExpect(status().isOk())
        .andExpect(content().contentType("application/pdf"))
        .andExpect(header().string("Content-Disposition", containsString("historia-clinica-")))
        .andReturn();

    byte[] cuerpo = resultado.getResponse().getContentAsByteArray();
    assertThat(cuerpo.length).isGreaterThan(500);
    assertThat(new String(cuerpo, 0, 5, StandardCharsets.US_ASCII)).isEqualTo("%PDF-");
}
```

Código de Producción Implementado (AdminReporteController.java):

```
@GetMapping("/historias/{id}/pdf")
public ResponseEntity<byte[]> historiaPdf(@PathVariable Long id) {
    Optional<Mascota> mascota = historiaClinicaService.mascota(id);
    if (mascota.isEmpty()) {
        return ResponseEntity.notFound().build();
    }
    List<Cita> atenciones = historiaClinicaService.atenciones(id);
    byte[] pdf = historiaClinicaService.generarPdf(mascota.get(), atenciones);

    String nombre = mascota.get().getNombre().toLowerCase().replaceAll("[^a-z0-9]+", "-");
    return ResponseEntity.ok()
        .contentType(MediaType.APPLICATION_PDF)
        .header(HttpHeaders.CONTENT_DISPOSITION,
            "attachment; filename=\"historia-clinica-\" + nombre + "-" + id + ".pdf\"")
        .body(pdf);
}
```

B. Ciclo TDD y Evidencias

1. Fase Roja (RED):

- **Descripción:** Fase roja: 10 pruebas, 7 fallos. Pasaron las 2 de acceso y la prueba soloAtencionesDeLaMascota. Esta última fue un falso positivo: comprueba que un texto no aparezca, y como la ruta aún no existía, la página de error tampoco lo contenía. Se dejó registrado como una limitación de las pruebas negativas en fase roja.
- **Evidencia Visual:**

```

[ERROR] Tests run: 10, Failures: 7, Errors: 0, Skipped: 0, Time elapsed: 16.93 s <<< FAILURE! -- in com.dbodo...
[INFO] Results:
[ERROR] HU24HistoriaClinicaTest.buscaMascota:178 Model attribute 'mascotas'
[ERROR] HU24HistoriaClinicaTest.descargaPdf:237 Status expected:<200> but was:<404>
[ERROR] HU24HistoriaClinicaTest.listaMascotas:170 Model attribute 'mascotas'
[ERROR] HU24HistoriaClinicaTest.mascotaInexistente:254 Range for response status value 404 expected:<REDIRE...
[ERROR] HU24HistoriaClinicaTest.muestraAtenciones:201 No ModelAndView found
[ERROR] HU24HistoriaClinicaTest.muestraDatosDeMascotaYDueno:188 Status expected:<200> but was:<404>
[ERROR] HU24HistoriaClinicaTest.sinAtenciones:225 Status expected:<200> but was:<404>
[ERROR] Tests run: 10, Failures: 7, Errors: 0, Skipped: 0
[INFO] BUILD FAILURE
[INFO] Total time: 26.069 s
[INFO] Finished at: 2026-09-26T11:22:33-05:00

```

Extracto de evidencias\HU24\rojo\consola.txt (salida de mvnw test).

2. Fase Verde (GREEN):

- **Descripción:** Se agregó la dependencia OpenPDF, dos consultas con join fetch (conDueno en MascotaRepository e historialDeMascota en CitaRepository), HistoriaClinicaService, HistoriaClinicaServiceImpl, los métodos del controlador y dos plantillas. La prueba del PDF verifica el tipo de contenido, el nombre del archivo y que los primeros bytes sean la firma %PDF-. Resultado: 10 pruebas, 0 fallos; suite completa, 160 pruebas en verde.
- **Evidencia Visual:**

```

[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 8.461 s -- in com.dbodor.veterinariaw...
[INFO] Results:
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 16.734 s
[INFO] Finished at: 2026-09-26T11:28:17-05:00

```

Extracto de evidencias\HU24\verde\consola.txt (salida de mvnw test).

3. Fase de Refactorización (REFACTOR):

- **Descripción:** La entidad HistoriaClinica, con diagnóstico y observaciones, no se usa todavía porque ninguna pantalla la registra; corresponde al módulo del veterinario. La historia muestra por ahora los datos de la mascota y todas sus atenciones.

4. Hallazgos durante la Integración

Además de los defectos que cada ciclo rojo-verde detectó dentro de su historia, la integración del trabajo de los dos integrantes y el uso de la aplicación publicada revelaron los siguientes problemas.

N.º	Hallazgo	Cómo se detectó	Resolución
1	Al mover los enums del paquete model a enums, varias clases quedaron con imports rotos, incluidas las del mismo paquete que no necesitaban import.	Fallo de compilación con mvnw clean test-compile.	Reemplazo de imports y búsqueda de clases que usaban los enums sin import.
2	Al unir HU-16 con HU-19 y HU-20 fallaron 5 pruebas unitarias: la implementación consultaba findByEstadoOrderByNombreAsc mientras la prueba simulaba findByEstado, y los mensajes de error estaban sin tildes.	Suite completa: 70 pruebas, 5 fallos.	Se adaptó la implementación para cumplir ambas pruebas, sin modificar las pruebas del compañero; el orden alfabético se conservó ordenando en memoria.
3	El estado de una cita agendada se llamaba PENDIENTE en el panel y PROGRAMADA en HU-18 y HU-20.	Revisión del código antes de escribir las pruebas de HU-25.	Se unificó en PROGRAMADA y se ajustó el criterio de HU-25.
4	Para una cita completada se usaban FINALIZADA (al finalizar la cita) y ATENDIDA (en un reporte de ingresos por servicio que ninguna pantalla usa).	Revisión del código antes de escribir las pruebas de HU-26.	El reporte de HU-26 usa FINALIZADA. Queda pendiente corregir el método que usa ATENDIDA.
5	CitaServiceImpl y ProductoServiceImpl no tenían @Service, por lo que Spring no podía inyectarlos en los controladores.	Al integrar las pantallas de HU-25 y HU-22.	Se agregó la anotación sin cambiar su lógica.
6	Un cambio en la ruta de inicio del cliente (de /cliente/citas a /cliente/dashboard) rompió 10 aserciones en 9 clases de prueba.	Fase verde de HU-26 y suite completa.	Se actualizaron las aserciones. Se acordó correr la suite completa antes de cada pull request.
7	Falso positivo en la fase roja de HU-24: una prueba negativa pasó porque la ruta aún no existía.	Revisión de los resultados de la fase roja.	Documentado; la prueba es válida en fase verde.

N.º	Hallazgo	Cómo se detectó	Resolución
8	El campo de precio del formulario de productos tenía <code>step="100"</code> y el navegador solo aceptaba valores como 1, 101 o 201.	Uso real de la aplicación publicada.	Se corrigió a <code>step="0.01"</code> . MockMvc no ejecuta la validación HTML del navegador, por lo que este tipo de defecto requiere pruebas manuales o de extremo a extremo.
9	Las pantallas del cliente cargaban Tailwind desde <code>cdn.tailwindcss.com</code> ; en redes que no resuelven ese dominio se mostraban sin estilos.	Uso de la aplicación desde otra red (error <code>ERR_NAME_NOT_RESOLVED</code> en la consola del navegador).	El archivo se sirve desde el propio proyecto (<code>/js/tailwind.js</code>).

5. Conclusiones

El ciclo rojo-verde se aplicó en las once historias del bloque de administración, con 131 métodos de prueba de integración. Escribir las pruebas antes del código obligó a precisar los criterios de aceptación: varias reglas que en Jira estaban implícitas, como qué ocurre con una cita que empieza justo cuando termina otra o si la fecha de hoy es válida como nacimiento, quedaron resueltas como casos de prueba concretos.

La suite completa fue el mecanismo más útil durante la integración. Los defectos 2, 5 y 6 de la sección anterior no los habría detectado ninguna prueba aislada: aparecieron solo al ejecutar juntas las pruebas de los dos integrantes. Por eso se acordó ejecutar `mvnw clean test` antes de cada pull request.

Las pruebas de integración con MockMvc cubren seguridad, validación, servicio y persistencia, pero no el comportamiento del navegador. Los defectos 8 y 9 lo demuestran. Para la fase de pruebas cruzadas conviene complementar la suite con casos de prueba manuales sobre la aplicación publicada.